

GRAPH NEURAL NETWORKS

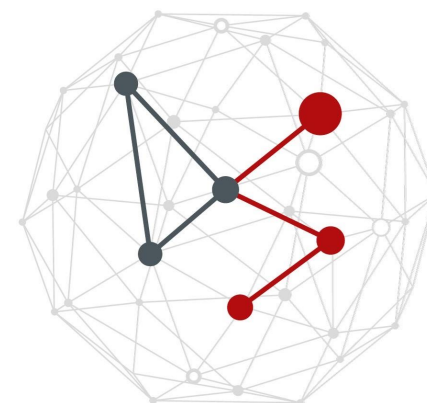
Michele Rossi

michele.rossi@unipd.it

Dept. of Information Engineering

Dept. of Mathematics

University of Padova, IT



GNN bibliographic sources

[Bishop2024] C. M. Bishop, H. Bishop, “Deep Learning: Foundations and Concepts,” Springer 2024.

[Ward2022] I. R. Ward, J. Joyner, C. Lickfold, Y. Guo, M. Bennamoun, “A Practical Tutorial on Graph Neural Networks,” ACM Computing Surveys, Vol. 54, No. 10, 2022.

[Hamilton2017] W. L. Hamilton et al., “Inductive Representation Learning on Large Graphs,” NIPS, Long Beach, CA, USA, 2017.

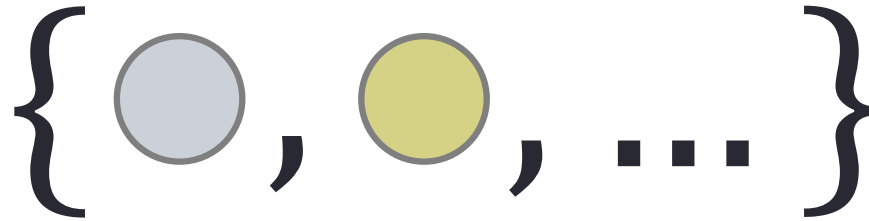
[Veličković2018] P. Veličković et al., “Graph Attention Networks,” ICLR, Vancouver, Canada, 2018.

[Cantürk-MLSP-2025] S. Cantürk, H. Shirzad, Q. Ya, G. Wolf, “Geometric and Topological Representation Learning,” Tutorial at the *IEEE Machine Learning for Signal Processing (MLSP)*, 2025, Istanbul/Turkey.

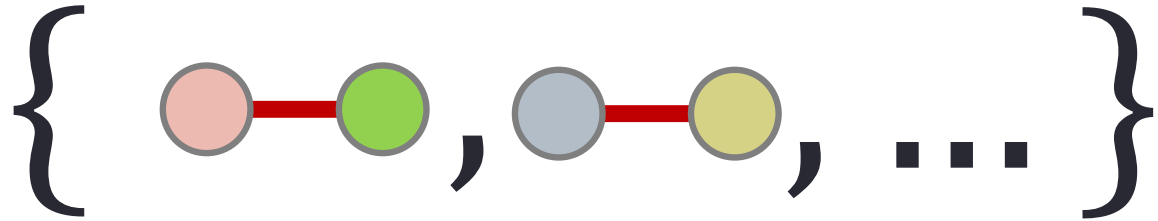
<https://2025.ieeemlsp.org/en/TUTORIALS.html>

Graphs

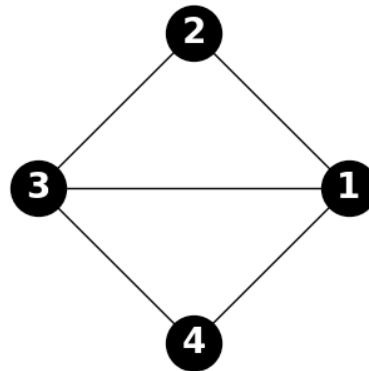
- Set of nodes V



- Set of edges E



- Adjacency matrix A



1	0	1	1	1
2	1	0	1	0
3	1	1	0	1
4	1	0	1	0
	1	2	3	4

Graphs

- **Graphs G** are defined by

- Vertices $V = \{1, 2, \dots, n\}$
- Edges (undirected) $E = \{E_k = (i, j) : i, j \in V\} \subseteq V \times V$
- Adjacency matrix $\mathbf{A} : A_{i,j} = 1 \Leftrightarrow (i, j) \in E$

- **Neighborhood** $\mathcal{N}(i) = \{j : (i, j) \in E \text{ or } (j, i) \in E\}$

Output degree Input degree

- **Degree** $d(i) = |\mathcal{N}(i)|$

Total degree = sum of **input + output degrees**

Graphs

- **Graphs G** are defined by

- Vertices $V = \{1, 2, \dots, n\}$
- Edges (undirected) $E = \{E_k = (i, j) : i, j \in V\} \subseteq V \times V$
- Adjacency matrix $\mathbf{A} : A_{i,j} = 1 \Leftrightarrow (i, j) \in E$

- **Neighborhood** $\mathcal{N}(i) = \{j : (i, j) \in E \text{ or } (j, i) \in E\}$

- **Degree** $d(i) = |\mathcal{N}(i)|$ $G = (V, E, \mathbf{A})$

- **Attributes**

- Node features $\mathbf{x} : V \rightarrow \mathbb{R}^s$
- Edge features $\mathbf{e} : E \rightarrow \mathbb{R}^{s'}$

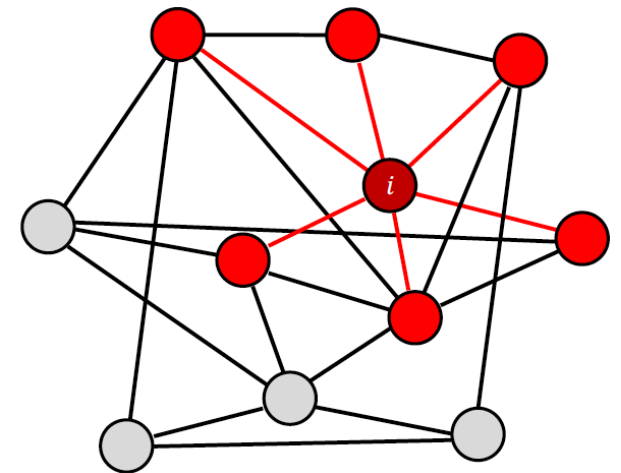
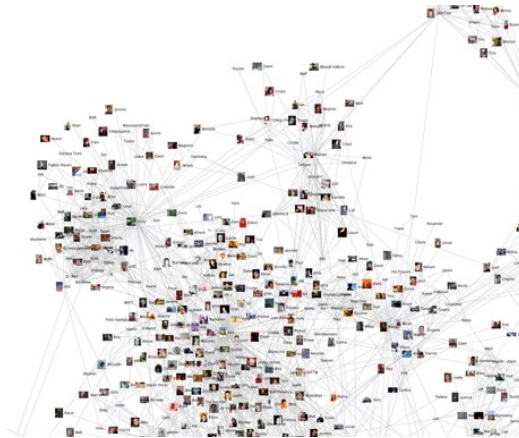
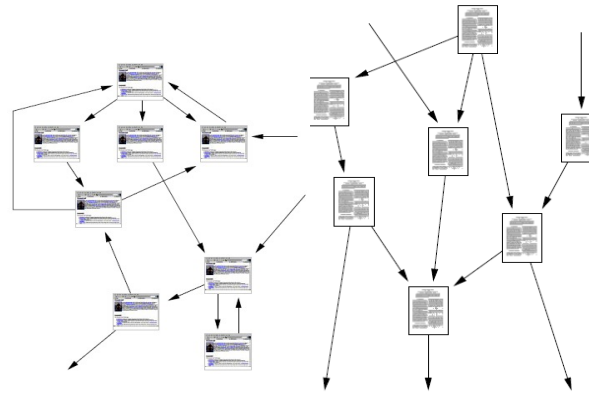


Illustration credit: [Bronstein et al. \(2021\)](#)

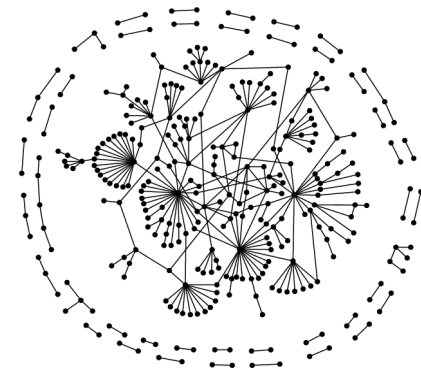
Abundance of graph structured data



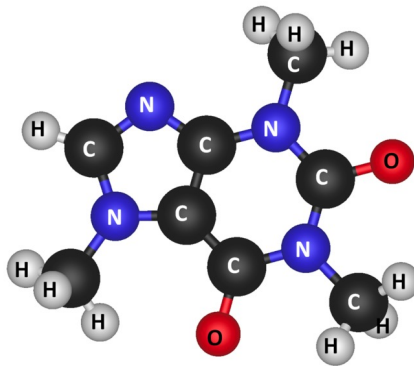
Social networks



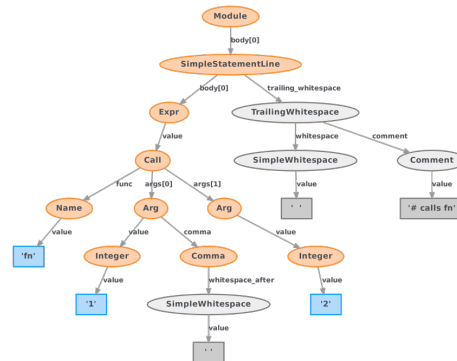
Knowledge graphs



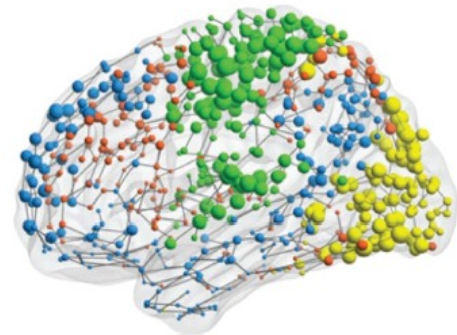
Biomedical networks



Molecular structure



The syntax of language and code



Networks of neurons

Tasks

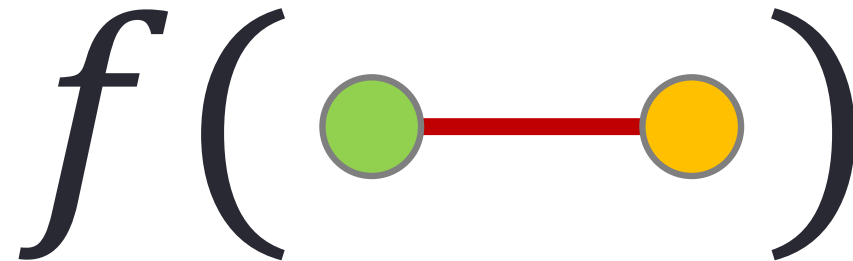
- Node level tasks

- Node classification
- Node clustering
- Node regression



- Edge level tasks

- Link prediction
- Edge classification
- Knowledge graph completion



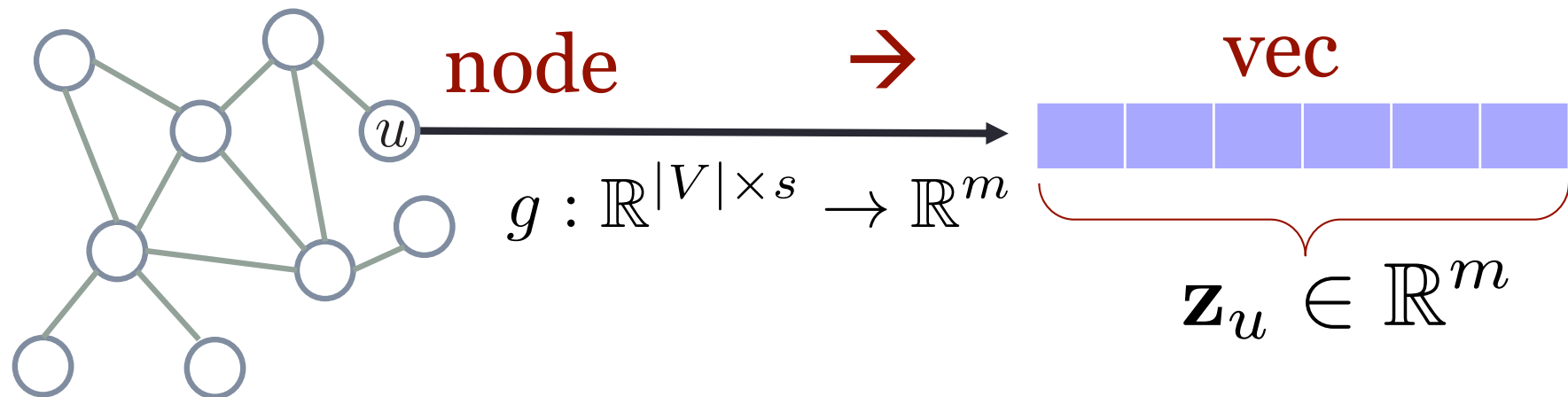
- Graph level tasks

- Graph classification
- Graph regression



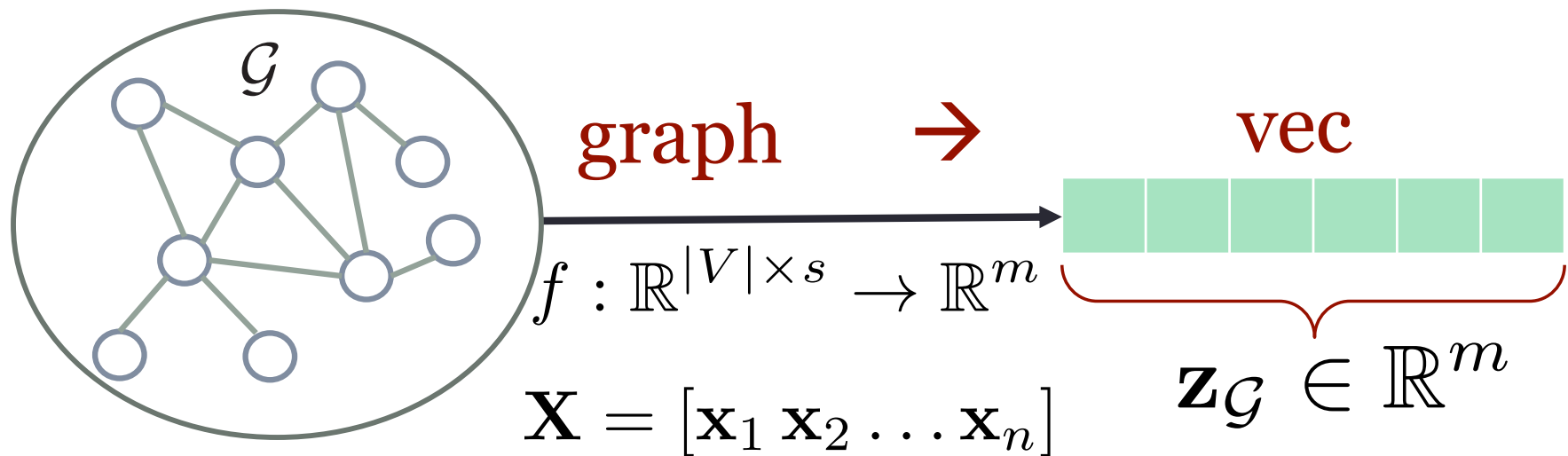
Representation learning

- **Goal:** to learn useful node and graph representations without resorting to hand-crafted features



Representation learning

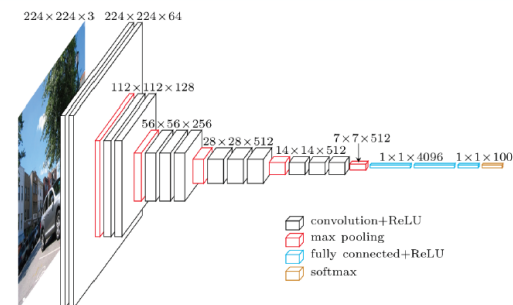
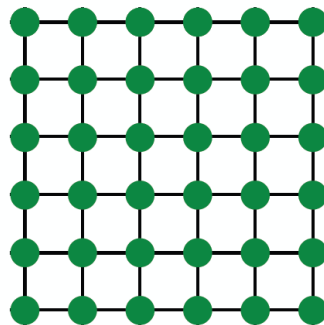
- **Goal:** to learn useful node and graph representations without resorting to hand-crafted features



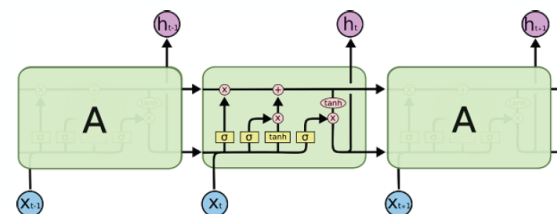
n nodes, $\mathbf{X}$ collects the feature vectors $\mathbf{x}_i$ from all nodes

Why is it non-trivial?

- Most deep learning methods are designed for **regular sequences** (i.e., grids in time, space or both)
 - **CNNs for regular grids** in 1D (audio), 2D (image), etc.



- **RNNs for temporal sequences:** texts, audio, etc.



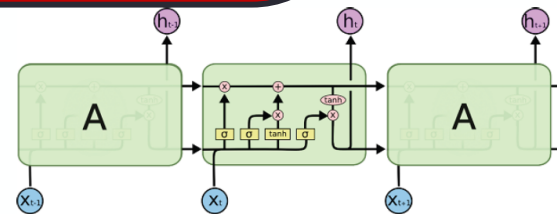
Why is it non-trivial?

- Most deep learning methods are designed for regular sequences (i.e., grids in time, space or both)
 - CNNs for regular grids in 1D (time), 2D (space), etc.

Graphs lack this regular grid structure!

How should we go about this?

- RNNs for time



7×512
 $1 \times 1 \times 4096$ $1 \times 1 \times 1000$
olution+ReLU
pooling
connected+ReLU
ax

Lack of structure example

E.g., predicting whether a given molecule is toxic:

- Molecules may have different numbers of atoms
- The atoms in a molecule may be of different types
- Each of these atoms may have different number of connections
- These connections can have different strengths

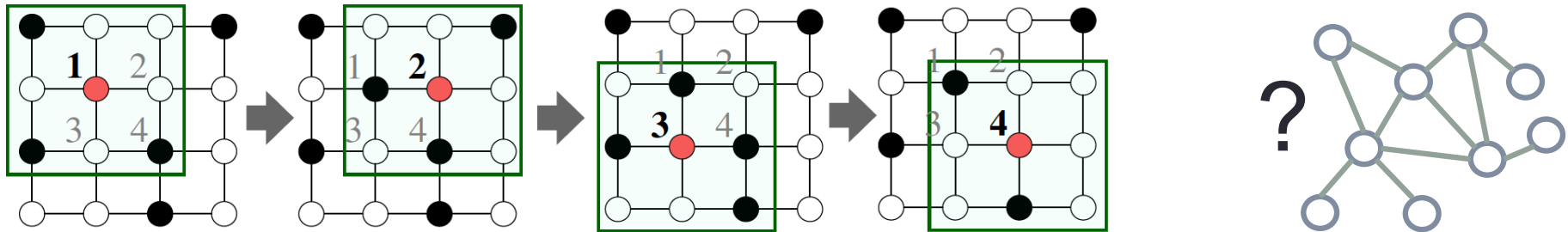
Representing graphs in a format that can be computed over is non-trivial, and the final representation chosen often depends significantly on the actual problem at stake

Ordering

- Graphs often **have no inherent ordering** present amongst the nodes
- Representing the graphs with vectors requires fixing an order of the nodes
- But what should we do **when the nodes have no inherent order?**

Key challenges with graph data

- 1) Complex geometric structure (often there is not straightforward notion of spatial locality)



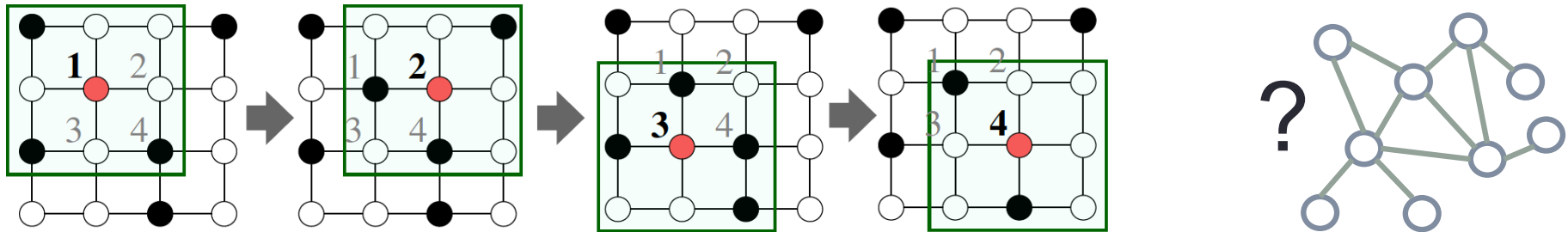
- 2) Node / edge ordering lead to **non-unique representations**

$$g : \mathbb{R}^{|V| \times s} \rightarrow \mathbb{R}^m \quad f(\mathbf{PAP}^T) = f(\mathbf{A}), \forall \mathbf{P} \in \mathbb{P}$$

Graph-level functions must be **invariant** to node permutations

Key challenges with graph data

- 1) Complex geometric structure (often there is not straightforward notion of spatial locality)



- 2) Node / edge ordering lead to **non-unique representations**

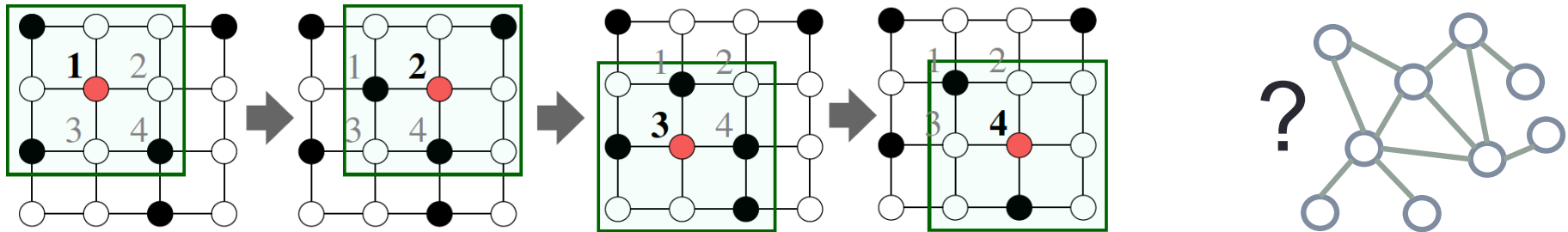
$$g : \mathbb{R}^{|V| \times s} \rightarrow \mathbb{R}^m \quad \boxed{f(\mathbf{PAP}^T)} = \boxed{f(\mathbf{A})}, \forall \mathbf{P} \in \mathbb{P}$$

Function on permuted adjacency matrix ($\mathbf{P}$ is permutation operator)

Function on original adjacency matrix

Key challenges with graph data

- 1) Complex geometric structure (often there is not straightforward notion of spatial locality)



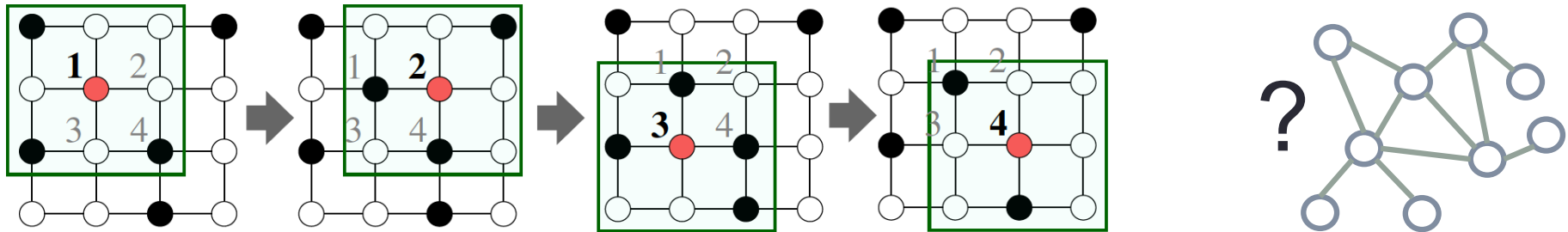
- 2) Node / edge ordering lead to **non-unique representations**

$$g : \mathbb{R}^{|V| \times s} \rightarrow \mathbb{R}^{|V| \times m} \quad g(\mathbf{PAP}^T) = \mathbf{P}g(\mathbf{A}), \quad \forall \mathbf{P} \in \mathbb{P}$$

Node-level functions must be **equivariant** to node permutations

Key challenges with graph data

- 1) Complex geometric structure (often there is not straightforward notion of spatial locality)



- 2) Node / edge ordering lead to **non-unique representations**

$$g : \mathbb{R}^{|V| \times s} \rightarrow \mathbb{R}^{|V| \times m} \quad \boxed{g(\mathbf{PAP}^T)} = \boxed{\mathbf{P}g(\mathbf{A})}, \forall \mathbf{P} \in \mathbb{P}$$

Permutation applied to input

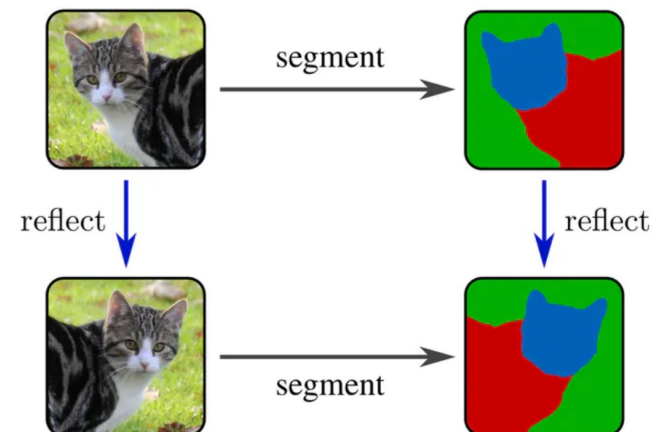
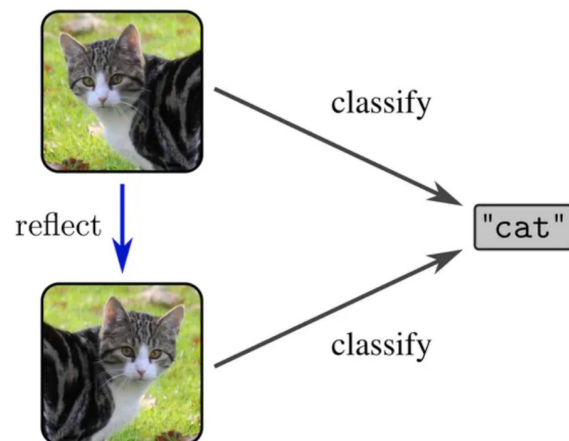
Equivalent permutation applied to output

Data invariance & equivariance

Invariance

Equivariance

Computer
Vision



Graph
Learning

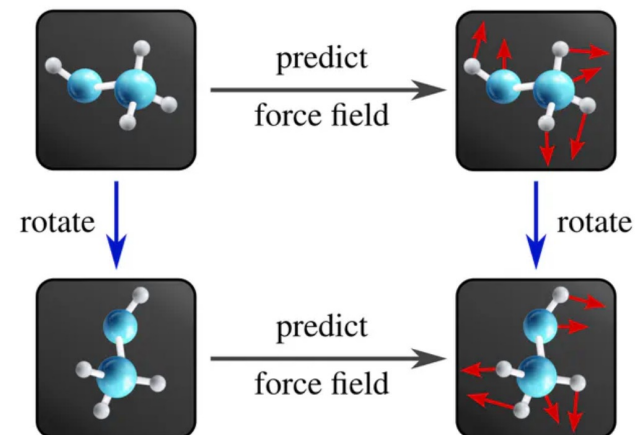
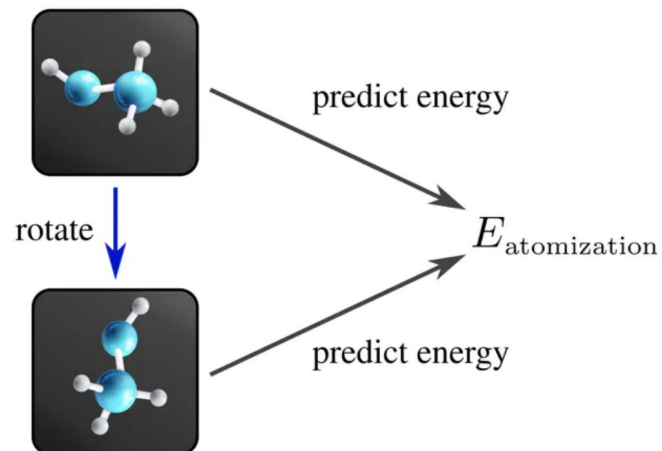


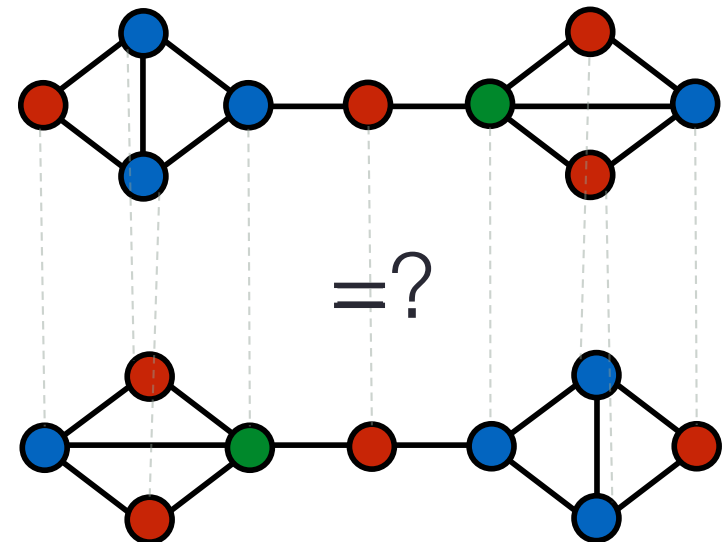
Illustration credit : [Maurice Weiler](#)

Graph isomorphism is a bottleneck

- Even determining whether two graphs are the same (*isomorphism testing*) **is a complex task**
- Best known teoretical run time $2^{O((\log n)^3)}$ with n vertices
- Known practical algorithms have *exponential run time* in the worst cases

$$\min_{\mathbf{P} \in \mathbb{P}} \|\mathbf{A}_1 - \mathbf{P}\mathbf{A}_2\mathbf{P}^T\| \stackrel{?}{=} 0$$

does there exist a permutation matrix
s.t. the adjacency matrix of graph one
equals a permutation of the adjacency
matrix of graph two?



Graph Neural Networks (GNN)

Objectives (part 1)

Desiderata – each **layer** should be

- A **highly flexible non-linear** function of the input
- **Differentiable** wrt its parameters
- Can be trained using **backpropagation** (stochastic gradient descent)
- Computations **should not depend on node representation order**

If the network is used to predict a graph-level property

- The **final layer** should be **invariant to any permutation of its input**

Graph Neural Networks (GNN)

Objectives (part 2)

Graph networks come in various sizes

- E.g., different molecules can have a different number of atoms, even within the same dataset / problem
- As such, fixed length representations (as used for FFNNs, CNNs) **are unsuitable**

Requirement

- The GNN should be capable of **handling variable length input**

Moreover, graphs can be very large

- E.g., social networks with millions of participants
- We also seek models that **scale well**

Graph Neural Networks (GNN)

Scaling Objectives (part 3)

Fixed number of parameters to learn per “layer”

- Independent of input and graph size

Computation and storage efficiency

- No more than $O(|E|+|V|)$ time & computation

Localization

- Acting on a local neighborhood of nodes

Modulation

- Able to specify arbitrary importance to different neighbors (the attention mechanism)

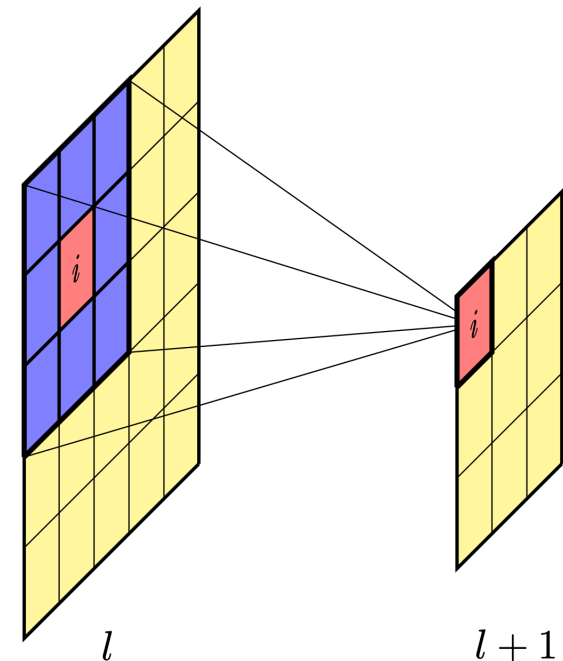
Seeking inspiration from CNNs

- **Convolutional filters**

- An image can be viewed as **graph structured data**: nodes are pixels and edges are pair of pixels that are adjacent in the image (diagonally, horizontally or vertically)
- In CNNs we make subsequent transformations using a so called **kernel** (in the figure a 3x3 blue patch)
- Output at layer $\ell + 1$

$$z_i^{\ell+1} = f \left(\sum_j w_j z_j^{(\ell)} + b \right)$$

$f()$: differentiable activation function, e.g., ReLU



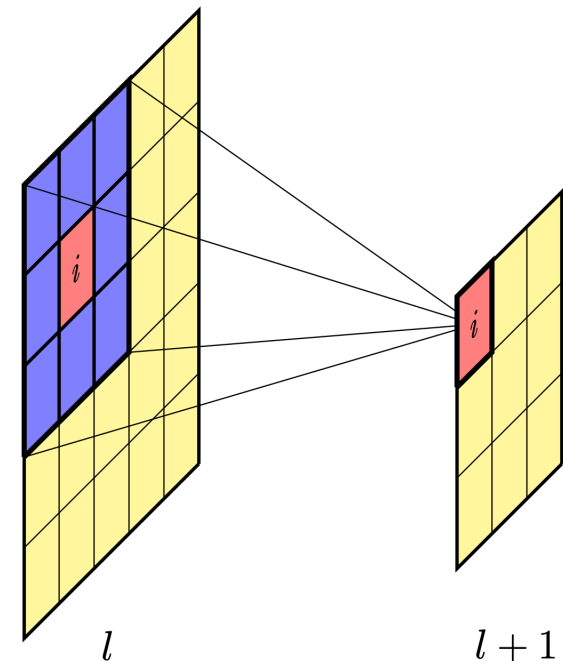
Seeking inspiration from CNNs

- **Convolutional filters**

- An image can be viewed as **graph structured data**: nodes are pixels and edges are pair of pixels that are adjacent in the image (diagonally, horizontally or vertically)
- In CNNs we make subsequent transformations using a so called **kernel** (in the figure a 3x3 blue patch)
- Output at layer $\ell + 1$

$$z_i^{\ell+1} = f \left(\sum_j w_j z_j^{(\ell)} + b \right)$$

Ordinary convolution (this Eq.) **is not order invariant**, as w_j is not invariant under permutations of its elements!



CONVOLUTION ON GRAPHS

Elements of Spectral Graph Theory

IDEA

- Represent the graph $G(V, E)$ through matrices
- Study the properties of such graph by investigating the *spectrum* of its matrix representation
- Key matrices are
 - Adjacency matrix **A**
 - Degree matrix **D** (of vertices)
 - Incidence matrix **B**

[Li-2022] Hanchen Li, “Properties and Applications of Graph Laplacians,” The University of Chicago Mathematics, REU 2022.

Definitions (1)

- Consider a Graph $G(V, E)$
- Set of vertices

$$V = \{1, 2, \dots, n\}$$

- Set of edges

$$E = \{E_1, E_2, \dots, E_{n(E)}\}$$

with $E_k = (i, j)$ with $i, j \in V$, $n(E) = |E|$

Graph signal: $x : V \rightarrow \mathbb{R}$

Can be collected in a vector (e.g., one value per node)

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T$$

Definitions (2)

Adjacency matrix (n x n)

$$A_{ij} = \begin{cases} 1 & (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

Degree (diagonal) matrix (n x n)

$$D_{ij} = \begin{cases} d(i) & i = j \\ 0 & \text{otherwise} \end{cases}$$

where $d(i)$ is the degree of node i

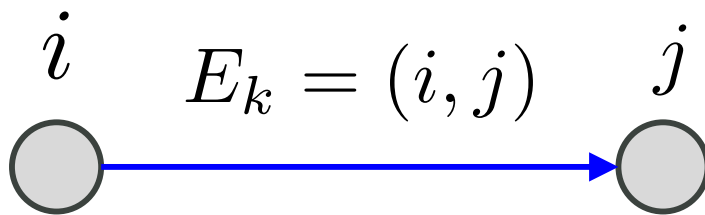
Definitions (3)

Incidence matrix ($n \times n(E)$)

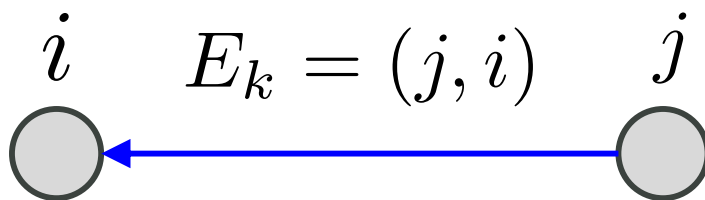
$$B_{ik} = \begin{cases} 1 & (i, j) \in E_k \text{ for some } j & \text{(edge leaving node } i) \\ -1 & (j, i) \in E_k \text{ for some } j & \text{(edge entering node } i) \\ 0 & \text{otherwise} \end{cases}$$

Remarks on **B**

- Nodes i is said to be incident to an edge E_k if it is one of its endpoints
- Nodes i and j are said to be both incident to an edge E_k if they are its two endpoints



$$B_{ik} = 1, B_{jk} = -1$$



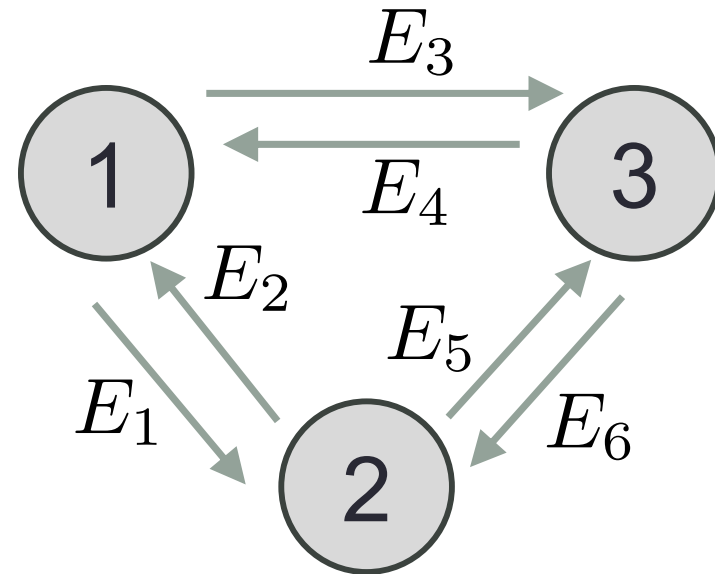
$$B_{ik} = -1, B_{jk} = 1$$

only 2 cases where both B_{ik} and B_{jk} are **non-zero!**
In both cases $B_{ik} \times B_{jk} = -1$

Incident matrix example

For each node (matrix row)

- +1 if link leaves the node
- -1 if link enters the node



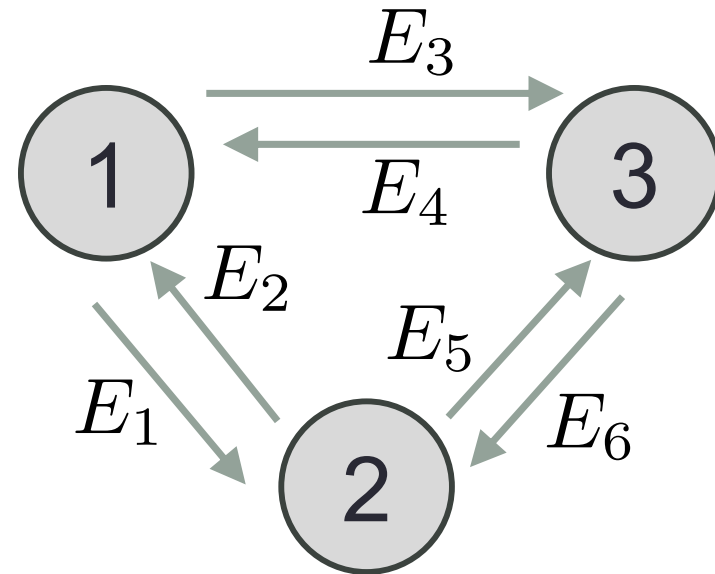
$$\mathbf{B} = \begin{array}{c} \begin{array}{cccccc} E_1 & E_2 & E_3 & E_4 & E_5 & E_6 \end{array} \\ \begin{bmatrix} +1 & -1 & +1 & -1 & 0 & 0 \\ -1 & +1 & 0 & 0 & +1 & -1 \\ 0 & 0 & -1 & +1 & -1 & +1 \end{bmatrix} \end{array} \begin{array}{c} 1 \\ 2 \\ 3 \end{array}$$

dimension is $n \times n(E) = 3 \times 6$

Incident matrix example

For each column

- 2-non zero entries (+1, -1)
- Associated with *source* and *destination* nodes for the link in question (column)



$$\mathbf{B} = \begin{bmatrix} E_1 & E_2 & E_3 & E_4 & E_5 & E_6 \\ +1 & -1 & +1 & -1 & 0 & 0 \\ -1 & +1 & 0 & 0 & +1 & -1 \\ 0 & 0 & -1 & +1 & -1 & +1 \end{bmatrix} \begin{matrix} 1 \\ 2 \\ 3 \end{matrix}$$

The Graph Laplacian $\mathbf{L}$

- Given a graph G (n nodes, with arbitrary ordering)
- 0/1 adjacency matrix $\mathbf{A}$
- We construct the **diagonal degree matrix $\mathbf{D}$** of G as:

$$D_{vv} = d(v) = \sum_u A_{vu} \quad \text{Degree of node } v \text{ is the number of edges incident at } v$$

- The **graph Laplacian** is a **square $n \times n$** matrix:

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

- It is the discrete analogous of the Laplacian operator from calculus

L vs B (1)

Proposition: The Laplacian matrix can equivalently be written as:

$$\mathbf{L} = \mathbf{B}\mathbf{B}^T$$

Proof.

$$(\mathbf{B}\mathbf{B}^T)_{ij} = \sum_k B_{ik} B_{jk} \quad \begin{array}{l} \text{dot product of } i\text{-th row of } \mathbf{B} \\ \text{and } j\text{-th column of } \mathbf{B}^T \end{array}$$

For $i=j$ the only non-zero contributions to this sum is for the edges that are incident to vertex i , i.e., for which either $B_{ik}=1$ (leaving node i) or $B_{ik}=-1$ (entering node i), in both cases their product is $+1$. The sum returns their number, i.e., the degree of node i . Hence

$$(\mathbf{B}\mathbf{B}^T)_{ii} = d(i)$$

L vs B (2)

Proposition: The Laplacian matrix can equivalently be written as:

$$\mathbf{L} = \mathbf{B}\mathbf{B}^T$$

Proof. (continued)

$$(\mathbf{B}\mathbf{B}^T)_{ij} = \sum_k B_{ik} B_{jk} \quad \text{dot product of } i\text{-th row of } \mathbf{B} \text{ and } j\text{-th column of } \mathbf{B}^T$$

For $i \neq j$, the term $(B_{ik} \times B_{jk})$ is **non-zero** iff both terms are non-zero, i.e., iff vertex i and vertex j are incident to the same edge k . In other words, if E_k has nodes i and j and its two endpoints. In this case this product is -1

L vs B (3)

Proposition: The Laplacian matrix can equivalently be written as:

$$\mathbf{L} = \mathbf{B}\mathbf{B}^T$$

Proof. (continued)

From the previous facts, it holds:

$$(\mathbf{B}\mathbf{B}^T)_{ij} = \begin{cases} \sum_{k|i \in E_k} 1 = d(i) & i = j \\ -1 & i \neq j, (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

which reveals that

$$\mathbf{B}\mathbf{B}^T = \mathbf{D} - \mathbf{A} = \mathbf{L}$$

QED

Theorem: $\mathbf{L}$ is positive semi-definite

We need to show that $\mathbf{x}^T \mathbf{L} \mathbf{x} \geq 0$, $\forall \mathbf{x} \in \mathbb{R}^n$

Proof.

From the previous equivalence relation:

$$\begin{aligned} \mathbf{x}^T \mathbf{L} \mathbf{x} &= \mathbf{x}^T (\mathbf{B} \mathbf{B}^T) \mathbf{x} \\ &= (\mathbf{B}^T \mathbf{x})^T (\mathbf{B}^T \mathbf{x}) \end{aligned}$$

Moreover, the k -th element of $\mathbf{B}^T \mathbf{x}$ is equal to

$$(\mathbf{B}^T \mathbf{x})_k = (x_i - x_j), \text{ with } (i, j) \in E_k$$

Theorem: $\mathbf{L}$ is positive semi-definite

We need to show that $\mathbf{x}^T \mathbf{L} \mathbf{x} \geq 0, \forall \mathbf{x} \in \mathbb{R}^n$

Proof.

It follows that:

$$\begin{aligned} \mathbf{x}^T \mathbf{L} \mathbf{x} &= \mathbf{x}^T (\mathbf{B} \mathbf{B}^T) \mathbf{x} \\ &= (\mathbf{B}^T \mathbf{x})^T (\mathbf{B}^T \mathbf{x}) \\ &= \sum_{(i,j) \in E} (x_i - x_j)^2 \longleftarrow \mathbf{a}^T \mathbf{a} = \|\mathbf{a}\|_2^2 \\ &\geq 0 \end{aligned}$$

QED

Some properties of L

- L is **real** and **symmetric**
 - All its **eigenvalues** are real
 - Its **eigenvectors** are orthogonalsee the almighty eigenvalue decomposition theorem...
- It descends that L is a **positive semi-definite** matrix
 - All its eigenvalues λ_i are non-negative, $\lambda_i \geq 0$
 - We can compute its *eigendecomposition* as

$$\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$$

- Diagonal matrix of **eigenvalues**

$$\mathbf{\Lambda} = \text{diag}(\lambda_0, \lambda_1, \dots, \lambda_{n-1})$$

- **U** is an **orthonormal** matrix containing the **eigenvectors** of **L** as columns

Some properties of L

- L is **real** and **symmetric**
 - All its **eigenvalues** are real
 - Its **eigenvectors** are orthogonal
(see eigenvalue decomposition theorem)
- L is a **positive semi-definite** matrix
 - All its eigenvalues λ_i are non-negative, $\lambda_i \geq 0$
 - We can compute its *eigendecomposition* as

$$\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$$

- As a side note, observe that

$$\begin{aligned}\mathbf{L}^i &= (\mathbf{U}\mathbf{\Lambda}\mathbf{U}^T)^i = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T\mathbf{U}\mathbf{\Lambda}\mathbf{U}^T \dots \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T = \\ &= \mathbf{U}\mathbf{\Lambda}^i\mathbf{U}^T\end{aligned}$$

Identity matrix

Graph Fourier Transform (GFT)

- We consider the **eigenvectors** $\{u_0, \dots, u_{n-1}\}$ as the **Fourier basis** on the **graph G** and the **eigenvalues** $\{\lambda_0, \dots, \lambda_{n-1}\}$ as the corresponding **graph frequencies** (*frequency spectrum*)
- We define the **Graph Fourier Transform (GFT)** of the spatial signal $\mathbf{x}$ as (projection onto the graph eigenspace)

$$\hat{\mathbf{x}} = \mathbf{U}^T \mathbf{x} \quad \text{GFT}$$

- And the **Inverse Graph Fourier Transform (IGFT)** as

$$\mathbf{x} = \mathbf{U} \hat{\mathbf{x}} \quad \text{IGFT}$$

Frequency components of $\mathbf{x}$

The entries

$$\hat{x}_i = \mathbf{u}_i^T \mathbf{x}$$

are the **frequency components** (or coefficients) of the signal $\mathbf{x}$ with respect to the basis function $\mathbf{u}_i$ and associated with the graph frequency λ_i

Graph convolution (1)

- The graph (temporal) convolution between a **filter $\mathbf{f}$** and **signal $\mathbf{x}$** is defined as

$$\mathbf{f} *_G \mathbf{x} = \mathbf{U}(\hat{\mathbf{f}} \odot \hat{\mathbf{x}}) = \mathbf{U}(\hat{\mathbf{f}} \odot \mathbf{U}^T \mathbf{x})$$



Graph

convolution in
time domain



element-wise
Hadamard product

convolution in
frequency domain =
product (convolution
theorem)

$$\hat{\mathbf{f}} \odot \hat{\mathbf{x}} = (\hat{f}_0 \hat{x}_0, \hat{f}_1 \hat{x}_1, \dots, \hat{f}_{n-1} \hat{x}_{n-1})^T$$

Graph convolution (2)

- Hence, it holds

$$\mathbf{f} *_G \mathbf{x} = \mathbf{U}(\hat{\mathbf{f}} \odot \mathbf{U}^T \mathbf{x})$$

- It is easier to express the Hadamard product as a **standard matrix-vector multiplication**, as follows

$$\hat{\mathbf{f}} \odot \mathbf{a} = \hat{\mathbf{F}} \mathbf{a}$$

with

$$(\hat{\mathbf{F}})_{ij} = (\text{diag}(\hat{\mathbf{f}}))_{ij} = \begin{cases} \hat{f}_i & i = j \\ 0 & \text{otherwise} \end{cases}$$

diagonal filter matrix

containing a vector of Fourier coefficients

Graph convolution (3)

- We can thus write

$$\mathbf{f} *_G \mathbf{x} = \mathbf{U}(\hat{\mathbf{f}} \odot \mathbf{U}^T \mathbf{x}) = \mathbf{U} \hat{\mathbf{F}} \mathbf{U}^T \mathbf{x}$$

- We need to design the diagonal matrix $\hat{\mathbf{F}}$

- **Problems**

- Such filters are non localized in space
 - Their learning complexity is $O(n)$
(the dimensionality of the data)
- These issues can be overcome using a polynomial filter

Graph convolution (3)

- We can thus write

$$\mathbf{f} *_G \mathbf{x} = \mathbf{U}(\hat{\mathbf{f}} \odot \mathbf{U}^T \mathbf{x}) = \mathbf{U} \hat{\mathbf{F}} \mathbf{U}^T \mathbf{x}$$

- We need to design the **diagonal matrix** $\hat{\mathbf{F}}$
- A common and convenient alternative is to use a **polynomial parameterization** on powers of the spectral matrix, i.e.,

$$\hat{\mathbf{F}}_w \approx \sum_{i=0}^d w_i \mathbf{\Lambda}^i$$

[Hammond-2011] David K. Hammond et al.. “Wavelets on graphs via spectral graph theory,” *Applied and Computational Harmonic Analysis*, 2011.

Graph convolution (4)

- It holds

$$\begin{aligned}\mathbf{f} *_G \mathbf{x} &= \mathbf{U} \hat{\mathbf{F}}_w \mathbf{U}^T \mathbf{x} = \sum_{i=0}^d w_i \mathbf{U} \boldsymbol{\Lambda}^i \mathbf{U}^T \mathbf{x} \\ &= \mathbf{U} \sum_{i=0}^d w_i \boldsymbol{\Lambda}^i \hat{\mathbf{x}} \triangleq \mathbf{U} g_{\mathbf{w}}(\boldsymbol{\Lambda}) \hat{\mathbf{x}}\end{aligned}$$

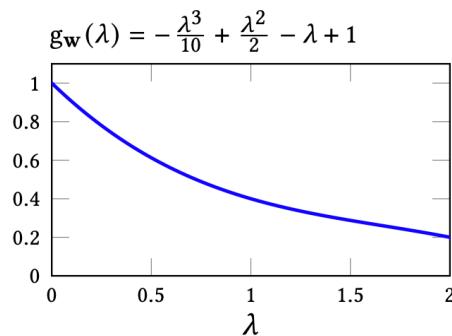
- Where $g_{\mathbf{w}}(\lambda)$ is a polynomial defining the filter (frequency) response as a function of the graph frequencies λ_i
- The design of this polynomial is independent of the graph structure (the Laplacian)

Graph convolution (5)

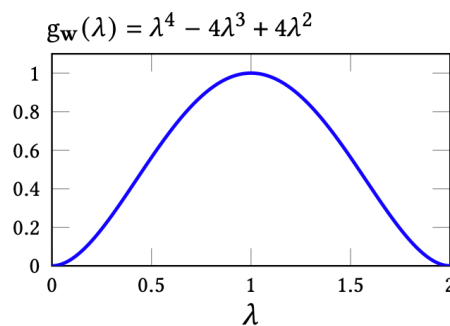
- From the previous slide the graph convolution in the frequency domain is

$$\hat{\mathbf{y}} \triangleq \mathbf{U}^T (\mathbf{f} *_G \mathbf{x}) = g_{\mathbf{w}}(\Lambda) \hat{\mathbf{x}}$$

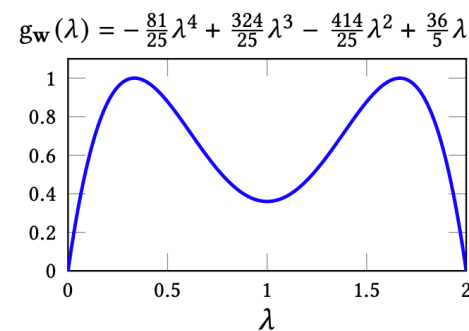
- Polynomial approximations for four common graph filters



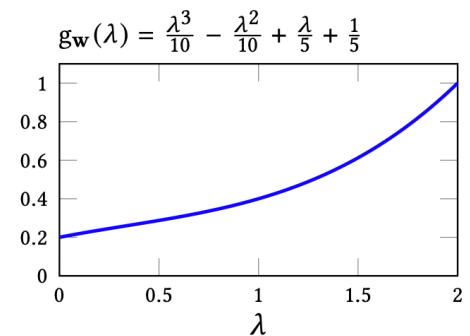
(a) Low-pass



(b) Band-pass



(c) Comb-pass



(d) High-pass

Graph convolution (6)

- It also holds

$$\begin{aligned}\mathbf{f} *_G \mathbf{x} &= \mathbf{U} \hat{\mathbf{F}}_w \mathbf{U}^T \mathbf{x} = \sum_{i=0}^d w_i \mathbf{U} \Lambda^i \mathbf{U}^T \mathbf{x} \\ &= \sum_{i=0}^d w_i (\mathbf{U} \Lambda \mathbf{U}^T)^i \mathbf{x} = \sum_{i=0}^d w_i \mathbf{L}^i \mathbf{x}\end{aligned}$$

- Recalling the decomposition $\mathbf{L} = \mathbf{U} \Lambda \mathbf{U}^T$

Graph convolution (6bis)

$$\begin{aligned}\mathbf{f} *_G \mathbf{x} &= \mathbf{U} \hat{\mathbf{F}}_w \mathbf{U}^T \mathbf{x} = \sum_{i=0}^d w_i \mathbf{U} \mathbf{\Lambda}^i \mathbf{U}^T \mathbf{x} \\ &= \sum_{i=0}^d w_i (\mathbf{U} \mathbf{\Lambda} \mathbf{U}^T)^i \mathbf{x} = \sum_{i=0}^d w_i \mathbf{L}^i \mathbf{x}\end{aligned}$$

Remark: this convolution Eq. does not require decomposing matrix $\mathbf{L}$, which is a big advantage for very large networks (complexity $\mathcal{O}(n^3)$)

Recap (1)

- Using $\mathbf{L}$, we can build **polynomials** of the form:

$$p_w(\mathbf{L}) = w_0 \mathbf{I}_n + w_1 \mathbf{L} + w_2 \mathbf{L}^2 + \cdots + w_d \mathbf{L}^d = \sum_{i=0}^d w_i \mathbf{L}^i$$

- Which is again **an nxn matrix** and can be alternatively represented by its vector of coefficients

$$\mathbf{w} = [w_0, w_1, \cdots, w_n]$$

- These polynomials can be thought of as the equivalent filters in a CNN and $\mathbf{w}$ are the weights of such filters

Recap (2)

- It follows that the **convolution** of vector $\mathbf{x}$ with a **polynomial filter** is defined as

$$\mathbf{x}' = p_{\mathbf{w}}(\mathbf{L})\mathbf{x}$$

Convolution examples

- $w_0=1$ and all remaining coefficients are zero

$$\mathbf{x}' = p_{\mathbf{w}}(\mathbf{L})\mathbf{x} = \sum_{i=0}^d w_i \mathbf{L}^i \mathbf{x} = w_0 \mathbf{I}_n \mathbf{x} = \mathbf{x}$$

- $w_1=1$ and all remaining coefficients are zero

$$(\mathbf{x}')_v = (\mathbf{L}\mathbf{x})_v = \sum_{u \in G} L_{vu} x_u$$

$$= \sum_{u \in G} (D_{vu} - A_{vu}) x_u = (\mathbf{D}\mathbf{x})_v - \sum_{u \in \mathcal{N}(v)} x_u$$

$$= d(v)x_v - \sum_{u \in \mathcal{N}(v)} x_u$$

The features of each node are combined with the features of its immediate neighborhood

Convolution examples

- $w_0 = 1$ and all remaining coefficients are zero

$$\mathbf{x}' = p_{\mathbf{w}}(\mathbf{L})\mathbf{x} = \sum_{i=0}^d w_i \mathbf{L}^i \mathbf{x} = w_0 \mathbf{I}_n \mathbf{x} = \mathbf{x}$$

- $w_1 = 1$ and all remaining coefficients are zero

$$(\mathbf{x}')_v = (\mathbf{L}\mathbf{x})_v = \sum_{u \in G} L_{vu} x_u$$

$$= \sum_{u \in G} (D_{vu} - A_{vu}) x_u = (\mathbf{D}\mathbf{x})_v - \sum_{u \in \mathcal{N}(v)} x_u$$

$$= d(v)x_v - \sum_{u \in \mathcal{N}(v)} x_u$$

Remark: This filter is node order equivariant (the sum does not depend on the order of nodes)!

On the role of d (1)

Proposition: if shortest path distance

$$\text{dist}_G(v, u) > i \Rightarrow (\mathbf{L}^i)_{vu} = 0$$

Proof.

Assume $\text{dist}_G(v, u) > i$ **(A1)**

- 1) First note that $L_{v,u} = 0$ only if the two nodes are distinct vertices that are not connected, i.e., if $A_{vu} = 0$
- 2) By repeatedly expressing matrix multiplications via explicit sums, we write

$$(\mathbf{L}^i)_{vu} = \sum L_{vk_1} L_{k_1 k_2} \cdots L_{k_{i-1} u}$$

On the role of d (2)

Proof. (continued)

$$(\mathbf{L}^i)_{vu} = \sum L_{vk_1} L_{k_1 k_2} \cdots L_{k_{i-1} u}$$

By contradiction, we assume $(\mathbf{L}^i)_{vu} \neq 0$

But, this is only possible if at least one of the terms in the above sum is non-zero, i.e., there exist $k_1, k_2, \dots, k_{i-1}$ such that $L_{vk_1} \neq 0, L_{k_1 k_2} \neq 0, \dots, L_{k_{i-1} u} \neq 0$

After removing possibly repeated occurrences of the k_r 's, this implies the existence of a **path of length no longer than i from v to u**

This implies $\text{dist}_G(v, u) \leq i$ which **contradicts (A1)**

QED

On the role of d (3)

It holds that

$$\begin{aligned}x'_v &= (p_w(\mathbf{L})\mathbf{x})_{\overset{\text{element } v}{v}} = (\overset{\text{row } v}{p_w(\mathbf{L})})_v \mathbf{x} \\&= \sum_{i=0}^d w_i (\mathbf{L}^i)_v \mathbf{x} = \sum_{i=0}^d w_i \sum_{u \in G} (\mathbf{L}^i)_{vu} x_u \\&= \sum_{i=0}^d w_i \sum_{\substack{u \in G, \\ \text{dist}_G(v,u) \leq i}} (\mathbf{L}^i)_{vu} x_u\end{aligned}$$

due to the previous **proposition**

This means that these **convolutional filters are d localized**.
The convolution at each node v occurs with those nodes u
that are no **more than d hops away**

On the role of d (3)

It holds that

$$\begin{aligned} x'_v &= (p_w(\mathbf{L})\mathbf{x})_v = (p_w(\mathbf{L}))_v \mathbf{x} \\ &= \sum_{i=0}^d w_i (\mathbf{L}^i)_v \mathbf{x} = \sum_{i=0}^d w_i \sum_{u \in G} (\mathbf{L}^i)_{vu} x_u \\ &= \sum_{i=0}^d w_i \sum_{\substack{u \in G, \\ \text{dist}_G(v, u) \leq i}} (\mathbf{L}^i)_{vu} x_u \end{aligned}$$

due to the previous proposition

Also, their **learning complexity is $O(d)$** , with $d \ll n$, just as for convolutional filters (CNNs)

Node-order equivariance (1)

- Polynomial filters are node-order equivariant
- Let us take a permutation matrix $\mathbf{P}$
 - Permutation matrices are orthogonal 0/1 matrices, i.e.,

$$\mathbf{P}\mathbf{P}^T = \mathbf{P}^T\mathbf{P} = \mathbf{I}_n$$

- A function $f()$ is called node-order equivariant iff, for any permutation $\mathbf{P}$ it holds

$$f(\mathbf{P}\mathbf{x}) = \mathbf{P}f(\mathbf{x})$$

Node-order equivariance (2)

- When switching to the new node-order, the following quantities are modified as follows:

$$\mathbf{x} \rightarrow \mathbf{P}\mathbf{x}$$

$$\mathbf{L} \rightarrow \mathbf{P}\mathbf{L}\mathbf{P}^T$$

$$\mathbf{L}^i \rightarrow (\mathbf{P}\mathbf{L}\mathbf{P}^T)^i = \mathbf{P}\mathbf{L}^i\mathbf{P}^T$$

Remark

$$(\mathbf{P}\mathbf{L}\mathbf{P}^T)^2 = \mathbf{P}\mathbf{L}\mathbf{P}^T\mathbf{P}\mathbf{L}\mathbf{P}^T = \mathbf{P}\mathbf{L}^2\mathbf{P}^T$$

- We define our function $f()$ as:

$$f(\mathbf{x}) = p_w(\mathbf{L})\mathbf{x}$$

Node-order equivariance (3)

- The following holds:

$$\begin{aligned} f(\mathbf{P}\mathbf{x}) &= \sum_{i=0}^d w_i (\mathbf{P}\mathbf{L}^i \overset{\text{Identity matrix}}{\mathbf{P}^T})(\mathbf{P}\mathbf{x}) = \\ &= \mathbf{P} \sum_{i=0}^d w_i \mathbf{L}^i \mathbf{x} \\ &= \mathbf{P} f(\mathbf{x}) \end{aligned}$$

- Which proves that convolution filters on graphs are **node-order equivariant**

How to build GNNs

- We can **stack polynomial filters** to build a Graph Neural Network (GNN)
- We define **K filter layers**, each with its own weights $\mathbf{w}^{(k)}$

STEPS

- 1) Initial node embeddings $\mathbf{h}^{(0)} = \mathbf{x}$
- 2) **Iterate for k = 1 to K**

$$p^{(k)} = p_{w^{(k)}}(\mathbf{L}) \quad \text{nxn filter matrix}$$

$$g^{(k)} = p^{(k)} \mathbf{h}^{(k-1)} \quad \text{matrix to vector multiplication}$$

$$\mathbf{h}^{(k)} = \sigma(g^{(k)}) \quad \text{non-linearity}$$

Chebyshev polynomials (1)

- More sophisticated graph filters can be defined, a popular one is from Chebyshev polynomials,

$$T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x), \text{ with: } T_0(x) = 1, T_1(x) = x, x \in \mathbb{R}$$

- The graph filter is

$$g_{\mathbf{w}}(\Lambda) = \sum_{i=0}^d w_i T_i(\Lambda)$$

[Defferrard-2016] M. Defferrard, X. Bresson, P. Vandergheynst. "Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering," NIPS, Barcelona, Spain, 2016.

Chebyshev polynomials (2)

- Note that

$$\mathbf{f} *_G \mathbf{x} = \mathbf{U} g_w(\mathbf{\Lambda}) \hat{\mathbf{x}} = \sum_{i=0}^d w_i \mathbf{U} T_i(\mathbf{\Lambda}) \mathbf{U}^T \mathbf{x} \stackrel{?}{=} \sum_{i=0}^d w_i T_i(\mathbf{L}) \mathbf{x}$$

- See [Appendix A](#) for a proof of the last equality (?).

Normalizing the Laplacian (1)

- We define:
- $$D^{-1/2} = \begin{bmatrix} \frac{1}{\sqrt{d(1)}} & 0 & \cdots & 0 \\ 0 & \frac{1}{\sqrt{d(2)}} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{\sqrt{d(n)}} \end{bmatrix}$$

- The **normalized adjacency matrix** is

$$\tilde{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$$

- The **normalized Laplacian** is

$$\tilde{\mathbf{L}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2} = \mathbf{I} - \tilde{\mathbf{A}}$$

Normalizing the Laplacian (2)

- The Laplacian matrix is often normalized
- It can be proven that
 - the eigenvalues of $\tilde{\mathbf{A}}$ are contained in $[-1,1]$
 - the eigenvalues of $\tilde{\mathbf{L}}$ are contained in $[0,2]$
- Normalization is preferable to
 - **Balance node influence:** with an unnormalized Laplacian, a node with a high degree will have a large value in the corresponding row and column. This can cause the node features to dominate the aggregated information from its neighbors, by biasing the learning process only towards such nodes
 - **Improve numerical stability:** normalization makes the Laplacian matrix scale-invariant, improving numerical stability during training. Especially for graphs with a large range of node degrees

Normalizing the Laplacian (3)

- The **normalized Laplacian** matrix is defined as

$$\tilde{\mathbf{L}}_{ij} = \begin{cases} 1 & \text{if } i = j \\ -\frac{1}{\sqrt{d(i)}\sqrt{d(j)}} & \text{if } i \neq j \text{ and } ((i, j) \in E \text{ or } (j, i) \in E) \\ 0 & \text{otherwise} \end{cases}$$

Towards modern GNNs

- We have seen that, by taking $w_1=1$ and $w_j=0$ (for $j \neq 1$)

$$p_w(\tilde{\mathbf{L}}) = \tilde{\mathbf{L}}$$

And the following holds

$$x'_v = x_v - \sum_{u \in \mathcal{N}(v)} \frac{x_u}{\sqrt{d(u)}\sqrt{d(v)}}$$

- 1) **aggregation** over immediate neighbor features
- 2) **combination** with the node's own features

Towards modern GNNs

- We have seen that, by taking $w_1=1$ and $w_j=0$ (for $j \neq 1$)

$$p_w(\tilde{\mathbf{L}}) = \tilde{\mathbf{L}}$$

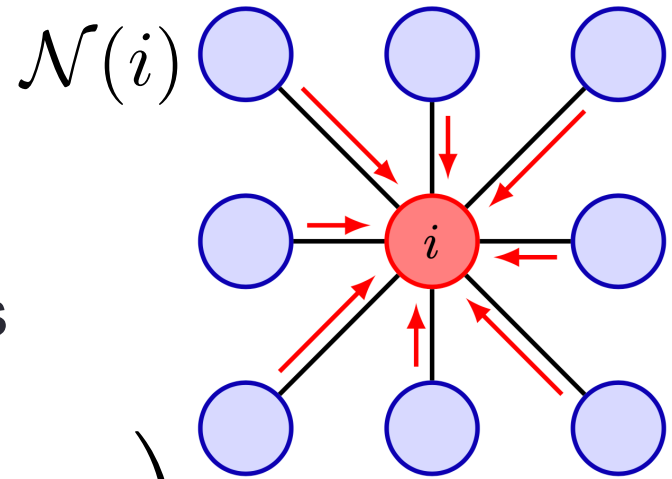
And the following holds

$$x'_v = x_v - \sum_{u \in \mathcal{N}(v)} \frac{x_u}{\sqrt{d(u)}\sqrt{d(v)}}$$

IDEA: what if we consider different kinds of aggregation and combination steps? Beyond what is possible with polynomial filters?

Convolution on graphs

- We separate node i from its neighbors



$$z_i^{(\ell+1)} = f \left(w_{\text{neigh}}^{(\ell)} \sum_{j \in \mathcal{N}(i)} z_j^{(\ell)} + w_{\text{self}}^{(\ell)} z_i^{(\ell)} + b \right)$$

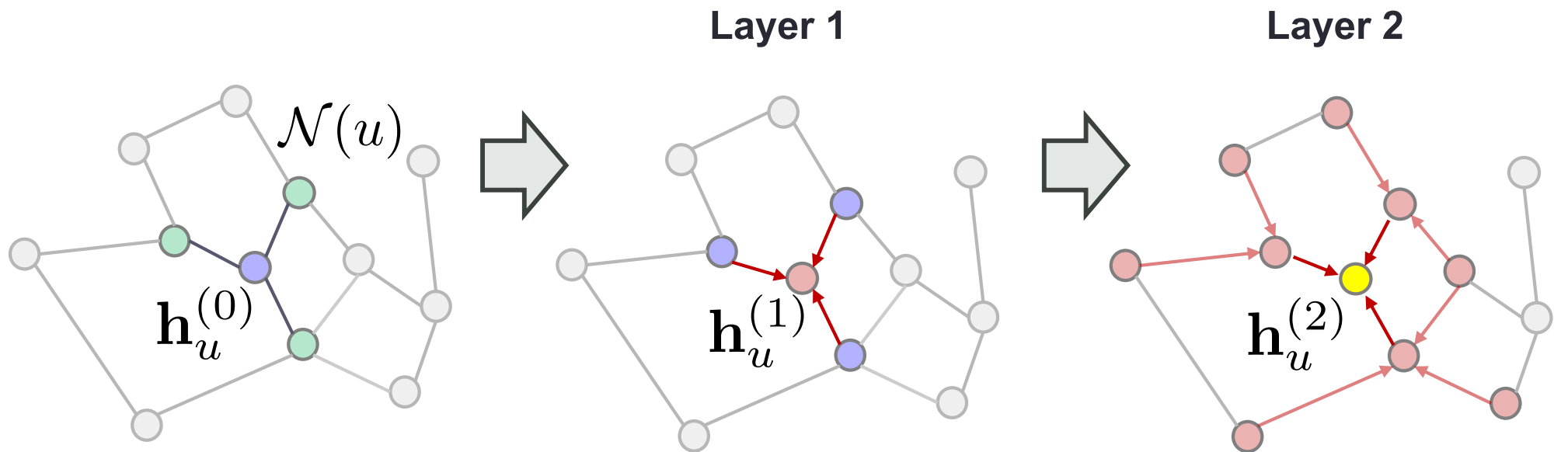
- **Weight parameters** are shared across all nodes: the information from the neighboring nodes is aggregated via a sum $\rightarrow$ this is **invariant** wrt any ordering of the nodes
- This operation is synchronously repeated for all nodes in the graph, if the **weights are the same for all nodes**, the results remain **unchanged under node reordering**

[Scarselli-2009] Scarselli et al., “The Graph Neural Network model,” IEEE Transactions on Neural Networks, Jan 2009.

MESSAGE PASSING ON NEURAL NETWORKS

Message Passing on Graphs

- Most GNNs are based on the idea of **iterative neighborhood aggregation**, a.k.a., **message passing**



$$\mathbf{h}_u^{(1)} := f_1(\mathbf{h}_u^{(0)}, \mathbf{h}_v^{(0)} \in \mathcal{N}(u)) \quad \mathbf{h}_u^{(2)} := f_1(\mathbf{h}_u^{(1)}, \mathbf{h}_v^{(1)} \in \mathcal{N}(u))$$

$\mathbf{h}_u^{(\ell)} \in \mathbb{R}^d$ embedding for node u at layer ℓ

Filtering node embeddings

- Let $\mathbf{h}_u^{(\ell)} \in \mathbb{R}^d$ be a d-dimensional column vector representing the embedding for node u at layer ℓ

We perform the filtering in two subsequent stages

Aggregation

- For each node u , we first aggregate the node embeddings from its neighborhood, combining them into a new vector $\mathbf{m}_u$ in a way that is permutation invariant

$$\mathbf{m}_u^{(\ell)} = \text{Aggregate}(\{\mathbf{h}_v^{(\ell)} : v \in \mathcal{N}(u)\})$$

Update

- We use another function to update the embedding vector at node u

$$\mathbf{h}_u^{(\ell+1)} = \text{Update}(\mathbf{h}_u^{(\ell)}, \mathbf{m}_u^{(\ell)})$$

The family of MPNNs

- **Message Passing Neural Networks (MPNNs)** are a general class of GNNs [Gilmer-2020]

The diagram illustrates the update equation for a node's embedding in a Message Passing Neural Network (MPNN). The equation is presented in two forms: a general form and a simplified form. The general form is $\mathbf{h}_u^{(\ell+1)} = \text{Update}^{(\ell)}\left(\mathbf{h}_u^{(\ell)}, \text{Aggregate}^{(\ell)}(\mathbf{h}_v^{(\ell)}, \forall v \in \mathcal{N}(u))\right)$. The simplified form is $\mathbf{h}_u^{(\ell+1)} = \text{Update}^{(\ell)}\left(\mathbf{h}_u^{(\ell)}, m_{\mathcal{N}(u)}^{(\ell)}\right)$. Annotations include: a blue box around $\mathbf{h}_u^{(\ell+1)}$ labeled 'Updated node embedding'; an orange box around the 'Update' function labeled 'Combined new embedding for this node from the previous layer'; a blue arrow pointing to $m_{\mathcal{N}(u)}^{(\ell)}$ labeled 'The "message" from neighbors'; and a blue arrow pointing to the 'Aggregate' function labeled 'Information aggregated from neighboring nodes'.

$$\mathbf{h}_u^{(\ell+1)} = \text{Update}^{(\ell)}\left(\mathbf{h}_u^{(\ell)}, \text{Aggregate}^{(\ell)}(\mathbf{h}_v^{(\ell)}, \forall v \in \mathcal{N}(u))\right)$$

Updated node embedding

$$\mathbf{h}_u^{(\ell+1)} = \text{Update}^{(\ell)}\left(\mathbf{h}_u^{(\ell)}, m_{\mathcal{N}(u)}^{(\ell)}\right)$$

Combined new embedding for this node from the previous layer

The "message" from neighbors

Information aggregated from neighboring nodes

[Gilmer-2020] J. Gilmer, "Message Passing Neural Networks," Book Chapter, *Springer Lecture Notes in Physics: Machine Learning meets Quantum Physics*, 2020.

Aggregation function (1)

- **Requirement:** It must only depend on the set of inputs and **NOT** on their ordering

- Simplest, most natural, choice

$$\text{Aggregate}(\{\mathbf{h}_v^{(\ell)} : v \in \mathcal{N}(u)\}) = \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(\ell)}$$

- **Possible drawback**
 - Gives stronger influence over those nodes that have a large number of neighbors compared to those with a few neighbors
- **Solution**
 - Normalize wrt the neighborhood size

Aggregation function (2)

- Sum with normalization

$$\text{Aggregate}(\{\mathbf{h}_v^{(\ell)} : v \in \mathcal{N}(u)\}) = \frac{1}{|\mathcal{N}(u)|} \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(\ell)}$$

- Drawback
 - This normalization discards information about the network structure and is provably less powerful than a plain sum [Kipf-2017] – so the choice depends on the relative importance of node features vs graph structure

[Kipf-2017] T. N. Kipf, M. Welling, “Semi-supervised classification with Graph Neural Networks,” ICLR, Toulon, France, 2017.

Aggregation function (3)

Sum with normalization – variation [Kipf-2017]

$$\text{Aggregate}(\{\mathbf{h}_v^{(\ell)} : v \in \mathcal{N}(u)\}) = \sum_{v \in \mathcal{N}(u)} \frac{\mathbf{h}_v^{(\ell)}}{\sqrt{|\mathcal{N}(u)| |\mathcal{N}(v)|}}$$

- Keeps into account the number of neighbors for each of the neighboring nodes, in an attempt of *recovering structure information in the embedding*

Aggregation function (4)

Sum + normalization + projection with dense linear layer

$$\text{Aggregate}(\{\mathbf{h}_v^{(\ell)} : v \in \mathcal{N}(u)\}) = \sum_{v \in \mathcal{N}(u)} \frac{\mathbf{W}\mathbf{h}_v^{(\ell)} + \mathbf{b}}{\sqrt{|\mathcal{N}(u)| |\mathcal{N}(v)|}}$$

- Keeps into account the number of neighbors for each of the neighboring nodes, learns a projection (can be normalized or not, normalization can be as in variant 2)

Aggregation function (5)

- One last, more powerful, variation

$$\text{Aggregate}(\{\mathbf{h}_v^{(\ell)} : v \in \mathcal{N}(u)\}) = \text{MLP}_\theta^{(\ell)} \left(\sum_{v \in \mathcal{N}(u)} \text{MLP}_\phi^{(\ell)}(\mathbf{h}_v^{(\ell)}) \right)$$

- The multi-layer perceptrons MLP_θ and MLP_ϕ have **shared weights** across layer ℓ
- This ensures **invariance** wrt the ordering of nodes
- This transformation represents a **universal approximator** for any permutation-invariant function

Convolution on graphs

- The aggregation operator that we have seen in the previous slides is called **convolution** on graphs
- It is the equivalent of the **convolution operator** (linear operation) in CNN → applied to a GNN context
- It goes under the names of
 - graph convolution,
 - neighborhood aggregation
- For each layer of the GNN, the weight matrices **W**, **b** for the projections (see aggregation variants 4, 5) are **shared among nodes**
- Note that, in CNNs kernel parameters (weights) are **shared across the input volume (image)**

The Update operator

- A simple form is

$$\text{Update}(\mathbf{h}_u^{(\ell)}, \mathbf{m}_u^{(\ell)}) = f(\mathbf{W}_{\text{self}}^{(\ell)} \mathbf{h}_u^{(\ell)} + \mathbf{W}_{\text{neigh}}^{(\ell)} \mathbf{m}_u^{(\ell)} + \mathbf{b}^{(\ell)})$$

where $f()$ is a nonlinear function, e.g., the **ReLU** function,
the weights $\mathbf{W}_{\text{self}}$, $\mathbf{W}_{\text{neigh}}$ and biases $\mathbf{b}$

- are **learnable** parameters
- are **shared** by all the nodes within the same layer

The above Eq. can be simplified by setting $\mathbf{W}_{\text{self}} = \mathbf{W}_{\text{neigh}} = \mathbf{W}$

$$\text{Update}(\mathbf{h}_u^{(\ell)}, \mathbf{m}_u^{(\ell)}) = f(\mathbf{W}^{(\ell)} (\mathbf{h}_u^{(\ell)} + \mathbf{m}_u^{(\ell)}) + \mathbf{b}^{(\ell)})$$

Message Passing in action

- Network initialization $\mathbf{h}_u^{(0)} = \mathbf{x}_u$

- Initial **node embeddings** stored into matrix

$$\mathbf{H}^{(0)} = \{\mathbf{h}_u^{(0)}\} = \mathbf{X} \text{ (data matrix)}$$

- Subsequent transformations via **L “stacked” layers**

$$\mathbf{H}^{(1)} = \mathbf{F}(\mathbf{X}, \mathbf{A}, \mathbf{W}^{(1)})$$

$$\mathbf{H}^{(2)} = \mathbf{F}(\mathbf{H}^{(1)}, \mathbf{A}, \mathbf{W}^{(2)})$$

$$\vdots = \vdots$$

$$\mathbf{H}^{(L)} = \mathbf{F}(\mathbf{H}^{(L-1)}, \mathbf{A}, \mathbf{W}^{(L)})$$

Remark

- Under a node reordering defined by the permutation matrix $\mathbf{P}$, the transformation computed by layer ℓ is **equivariant**, that is

$$\mathbf{PH}^{(\ell)} = \mathbf{F}(\mathbf{PH}^{(\ell-1)}, \mathbf{PAP}^T, \mathbf{W}^{(\ell)})$$

- As a consequence, the entire network (all stacked layers) computes an equivariant transformation

Node classification (1)

- A GNN is a series of layers each of which transforms set of node embeddings into a new set

$$\{\mathbf{h}_u^\ell\} \Rightarrow \{\mathbf{h}_u^{\ell+1}\}$$

- After the final layer L, we need to define:
 - a **readout (output) layer**
 - a **cost function** to train the network

Node classification (2) – readout

- Assume we want to classify a given **node** u into **C classes**
- The readout layer for this node is

$$\mathbf{y}_u = \text{Softmax} \left(\mathbf{W} h_u^{(L)} + \mathbf{b} \right), \mathbf{y}_u \in \mathbb{R}^C$$

- where $\mathbf{W} = \{\mathbf{w}_i\}$ and $\mathbf{b}$ are the parameters of a fully connected layer, in detail the above Eq. means:

$$y_{ui} = \frac{\exp(\mathbf{w}_i^T \mathbf{h}_u^{(L)} + b_i)}{\sum_{j=1}^C \exp(\mathbf{w}_j^T \mathbf{h}_u^{(L)} + b_j)}, i = 1, \dots, C$$

- which uses the embedding in the **last layer L**
- **Important:** the weights $\mathbf{W}$, $\mathbf{b}$ are **shared** across the nodes

Node classification (3) – loss

- At this point, we can define a **cross-entropy loss** function over the nodes and the number of classes

$$\mathcal{L} = - \sum_{u \in V_{\text{train}}} \sum_{i=1}^C t_{ui} \log(y_{ui})$$

- where $\{t_{ui}\}$ are the labels (target values) with a **one-hot encoding** for each node u
- **Remark:** since $\mathbf{W}$, $\mathbf{b}$ are shared among the nodes, the outputs y_{ui} are invariant with respect to permutation of the node ordering $\rightarrow$ the **loss is also invariant**

Edge (or link) prediction (1)

- Predicting the likelihood of a connection between any two nodes, a.k.a. “edge completion”
- Foundational, e.g., to predict a link in social networks or protein-to-protein interactions in biological networks
- Consider a pair of nodes (u, v)
- Common approaches for estimating the probability $\hat{Y}_{uv}$ of a link between them

Approach 1 – dot product + sigmoid

$$\hat{Y}_{uv} = \sigma((\mathbf{h}_u^{(L)})^T \mathbf{h}_v^{(L)})$$

Approach 2 – element-wise prod + dense layer (sigmoid out)

$$\hat{Y}_{uv} = \sigma(\mathbf{W} \cdot (\mathbf{h}_u^{(L)} \odot \mathbf{h}_v^{(L)}) + \mathbf{b})$$

Edge (or link) prediction (2)

Loss Function – binary cross-entropy loss

$$\mathcal{L} = - \sum_{(u,v) \in E_{\text{train}}} t_{uv} \log(\hat{Y}_{uv}) + (1 - t_{uv}) \log(1 - \hat{Y}_{uv})$$

- t_{uv} binary label (1 if link exists)
- E_{train} set of links whose label is known

Graph-level tasks (1)

- The goal is to **infer properties or labels of entire graphs**
- **Example:** predicting the toxicity of a chemical compound based on its molecular structure, represented as a graph
- This requires aggregating node embeddings $\mathbf{h}_u^{(L)}$ into a single graph representation
- This is achieved using a **readout function**
 - Also called **graph pooling**

$$\mathbf{h}_G = \text{Readout}(\mathbf{h}_u^{(L)} : u \in V)$$

Graph-level tasks (2)

- The **readout function**

$$\mathbf{h}_G = \text{Readout}(\mathbf{h}_u^{(L)} : u \in V)$$

- can be
 - the **sum**, the **mean**, **element-wise min** (or **max**) or **attention** pooling
- After that, the prediction for the graph is typically

$$\hat{\mathbf{Y}}_G = \mathbf{W}_G \cdot \mathbf{h}_G + \mathbf{b}_G \quad \text{linear projection}$$

Hence,

- **MSE** can be used for **graph regression**
- **Softmax + cross-entropy loss** can be used for **classification**

GNN advantages

Number of parameters is independent of the graph size

- The number of parameters in a GNN only depends on the dimension of the feature inputs

Inerently inductive

- After training, GNNs can be used to infer embeddings on unseen graphs

Naturally incorporate node features

- GNNs learns by aggregating node features over local neighborhoods

GRAPH ATTENTION NETWORKS

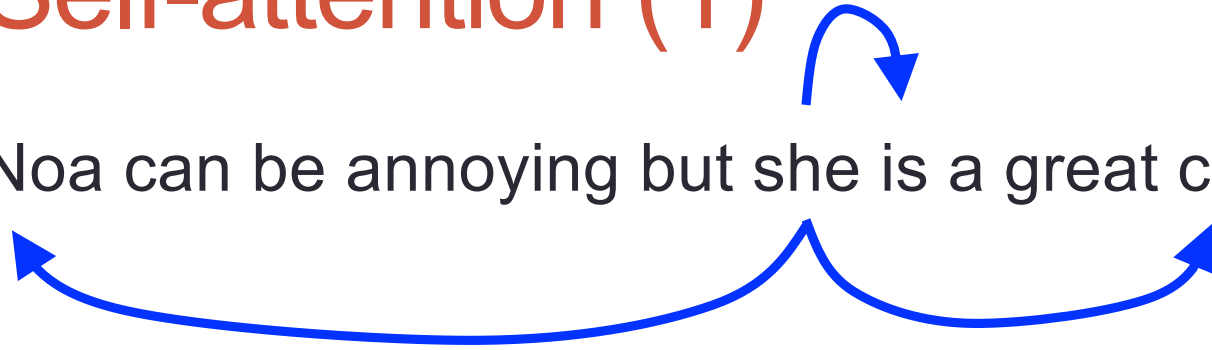
P. Velicković et al., “Graph attention networks,” ICLR 2018.

Attention: high level view

- Is a mechanism that determines the **importance** of each component in a sequence relative to any other component in the same sequence
- It is inspired by **attention in humans**, i.e., how humans perceive and process current information by selectively retrieving the most important details from the past, those that better explain or (correlate with) the present input
- **Example:** in [natural language processing](#), the importance of words is represented by [weights](#) assigned to each word in a sentence

Self-attention (1)

Noa can be annoying but she is a great cat



The word “she” in this sentence

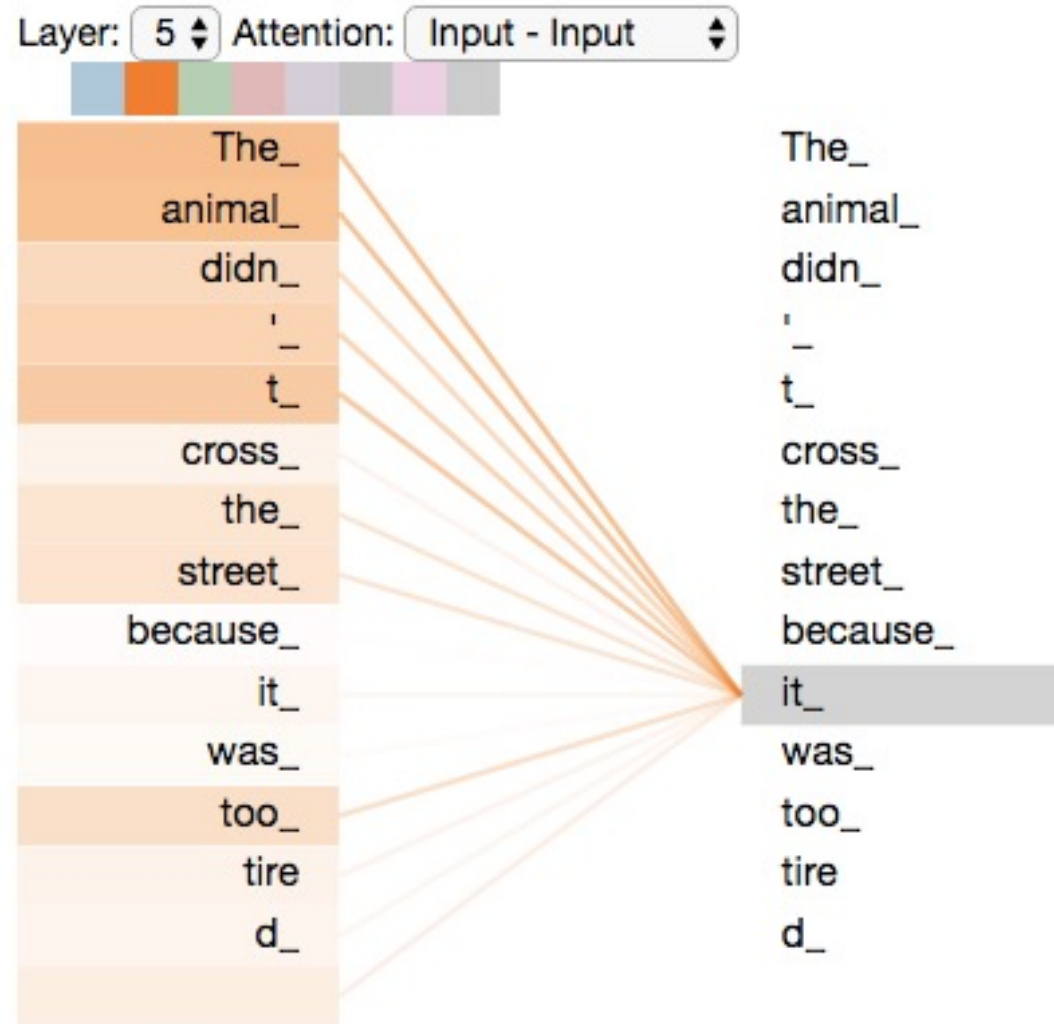
- Is semantically connected with the subject “Noa”
- Is connected with the noun “cat”
- And also with the state of being verb “is”

We would like to re-express words

- Contextualizing them wrt other words data items
- A proximity (position) based metrix cannot be the way
 - As shown in the example, words that are far apart may be correlated

Self-attention (2)

- Let's take the word **"it"** as the **target word** in a sentence
- A relation is built (correlation) between **each target word** and **any other word** in the same sentence
- This relation is stored in the form of a **probability vector** (softmax)
- A single word **attends** to multiple words in the same sentence (sequence)



Attention mechanism in neural nets

- At different decoding steps
 - Let the model focus on different parts of the input
 - Deciding **which source parts are important** for the task
 - This is implemented by computing a vector of **attention weights** over the input words (in general, over the **input embeddings**)
- The **attention mechanism**
 - Is part of a neural network
 - It is **jointly trained** with the other neural network weights
- It is the key ingredient in **transformers**
 - Can also be used in CNNs, RNNs, **GNNs**

Query, Keys and Values

- Terminology comes from **database management**
 - We have a **database of pictures** – each indexed via a **key**
 - We issue a **query** to search for a picture with specific content
 - The picture with **the best match** between query and key **is selected**
 - A **distribution (scores)** of the best matching pictures is returned

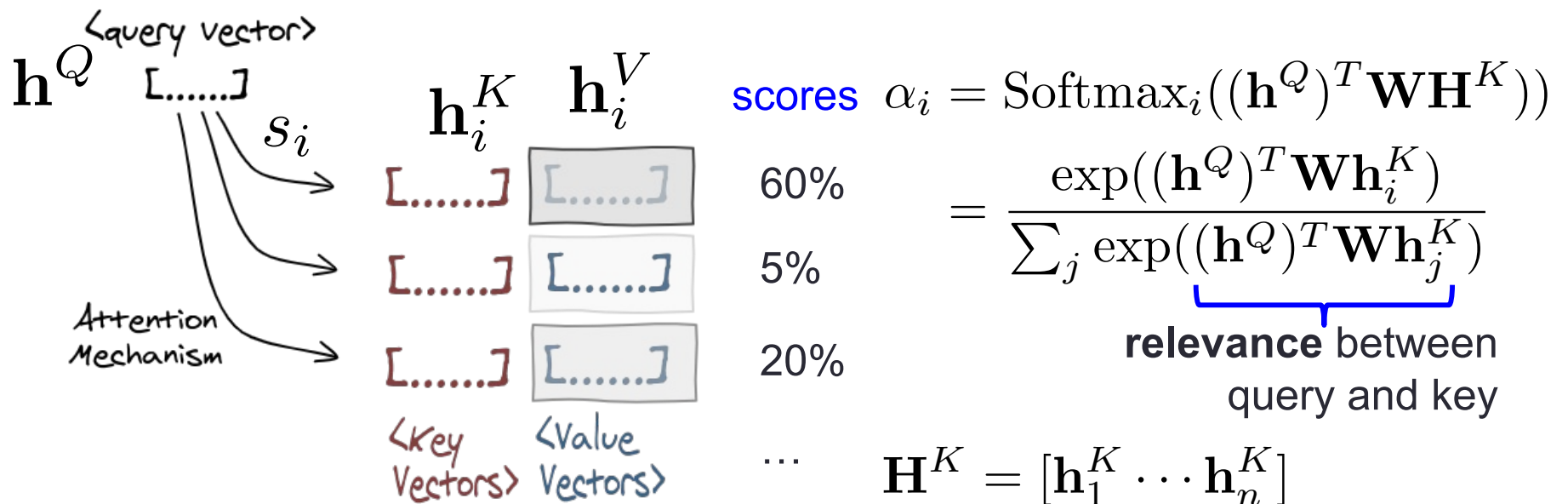


Figure credit: Nikhil Shah, “Attention? Another perspective,” 2020.

Graph attention network (GAT) (1)

- The **input** of the attention layer is a set of node features (embeddings)

$$\{\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_n\}, \mathbf{h}_i \in \mathbb{R}^d$$

- where n represents the number of nodes
- The **output** of the attention layer is another set of transformed node features

$$\{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n\}, \mathbf{z}_i \in \mathbb{R}^{d'}$$

- Possibly of different dimension d'

Graph Attention neTworks (GAT) (1)

- For each node u in the graph, the embedding vectors in its neighborhood are *aggregated* using **attention weights**

$$\mathbf{z}'_u = \text{Aggregate} \left(\{ \mathbf{h}_v^{(\ell)} : v \in (u \cup \mathcal{N}(u)) \} \right) = \sum_{v \in (u \cup \mathcal{N}(u))} \alpha_{uv} \mathbf{h}_v^{(\ell)}$$

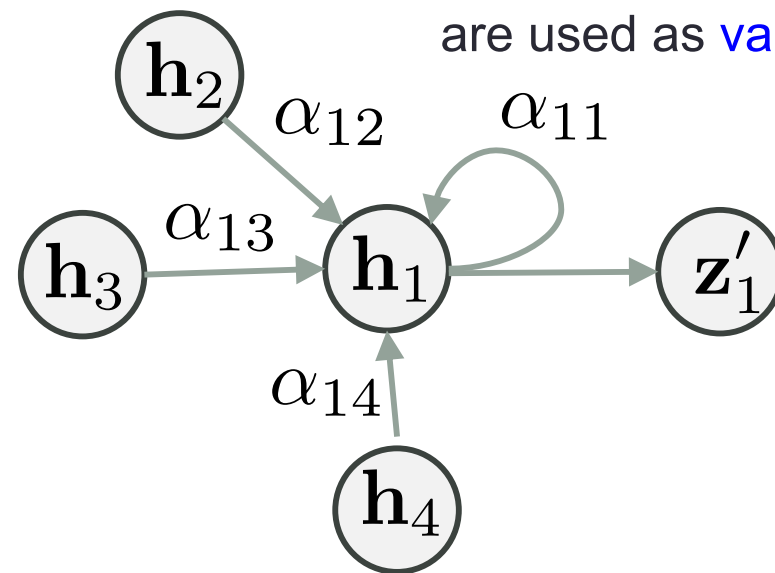


here embeddings
are used as **values**

with “attention” coefficients

$$\alpha_{uv} \geq 0$$

$$\sum_{v \in (u \cup \mathcal{N}(u))} \alpha_{uv} = 1$$



Graph attention networks (2)

- The **attention coefficients** can be computed as follows (**$\mathbf{W}$** is a square matrix of **learnable parameters**)

$$\alpha_{uv} = \frac{\exp(\mathbf{h}_u^T \mathbf{W} \mathbf{h}_v)}{\sum_{v' \in (u \cup \mathcal{N}(u))} \exp(\mathbf{h}_u^T \mathbf{W} \mathbf{h}_{v'})}$$



here embedding $\mathbf{h}_u$
acts as the **query**



here embedding $\mathbf{h}_v$
of the neighbors act
as the **key**

- A sort of **closeness (fitness)** is evaluated between the embedding of the **current node u** (query) and the embedding of its **neighbor v'** (key)
- The attention weight (score) α_{uv} is obtained via **softmax**

Graph attention networks (3)

- An alternative way of finding the **attention coefficients** is

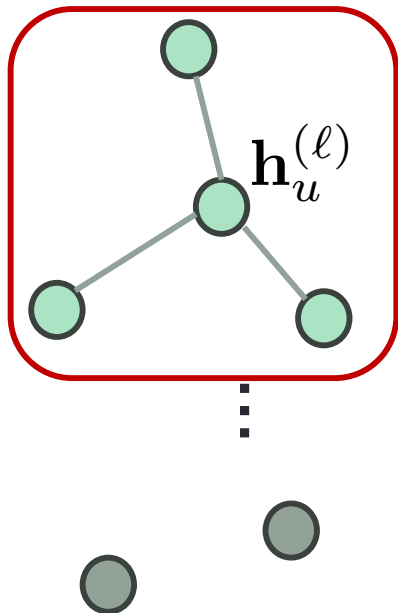
$$\alpha_{uv} = \frac{\exp(\text{MLP}(\mathbf{h}_u, \mathbf{h}_v))}{\sum_{v' \in (u \cup \mathcal{N}(u))} \exp(\text{MLP}(\mathbf{h}_u, \mathbf{h}_{v'}))}$$

- Where the MLP has a **single continuous output variable that must be invariant** if the input vectors are exchanged
- Moreover, provided that the MLP **is shared across all the nodes** in the network, the resulting aggregation with attention is equivariant under node reordering

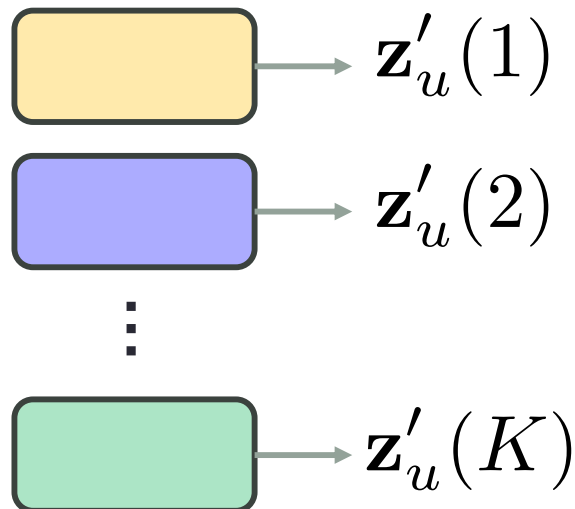
Multiple attention heads

- Multiple **attention heads** (as just presented) can be used in parallel, each with **independently learnable (attention) weights** (**not shared** among the multiple heads)

node embeddings
(one for each node)

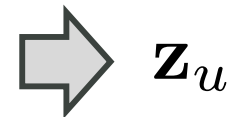


K attention heads
(independent,
running in parallel)



From here the attention layer **becomes equivalent to a transformer encoder**

concatenation
(or averaging)
and then **linear projection**



GAT advantages (1)

- High computation efficiency
 - The operations of self-attention can be parallelized across edges
 - The computation of output features can be parallelized across nodes
 - No eigendecomposition or computationally intensive operations on matrices are required
 - Multi-head attention imply a linear memory/computation increase by a factor of K , individual head computations are fully independent and can be parallelized
- Attention mechanism
 - Allows assigning different importance to nodes of a same neighborhood, enabling a leap in model capacity
 - Analyzing the learned attention weights can reveal structure/dependencies (explainability)

GAT advantages (2)

- The attention mechanism is applied in a **shared manner** to all edges in the graph and, therefore, does not depend on upfront access to the global graph structure
 - The graph **does not need to be undirected** → we may simply leave out computing A_{ij} if link $i \rightarrow j$ is not present
 - It makes attention applicable to **inductive learning**
 - i.e., including tasks where the model is evaluated on graphs that are completely unseen during training

APPLICATION EXAMPLE

Modeling sound signals

Task and Dataset

Task

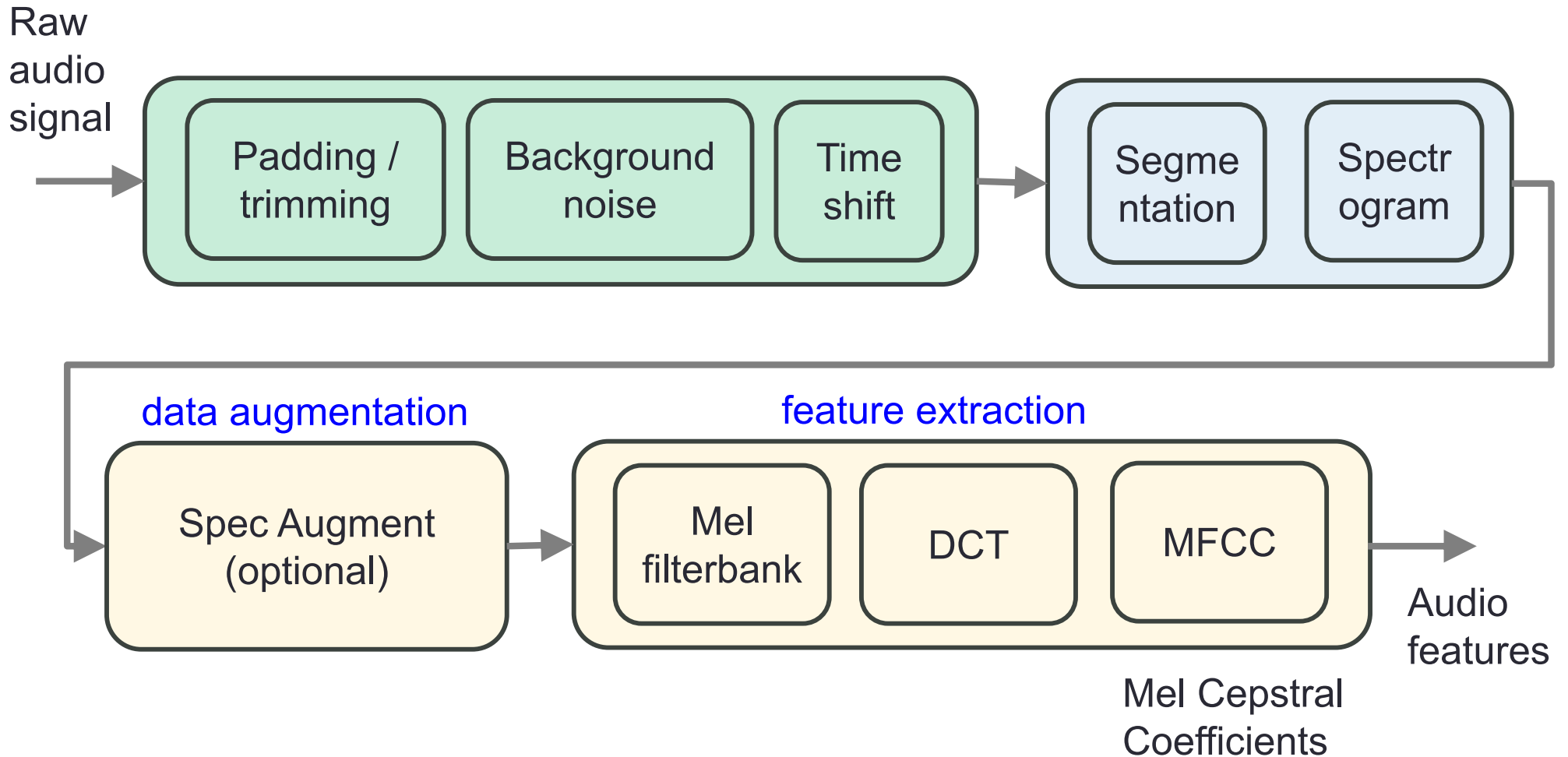
- Classify audio commands from raw audio data
- Also known as “keyword spotting”

Dataset

- Google Speech Commands Dataset v2
- Contains 105,829 single word utterances
- 35 commands, sampled at 16 kHz
- Diverse set of speakers
- 6 variants of background noise

Preprocessing (1)

- Standard audio preprocessing pipeline



Preprocessing (1)

- **Padding/trimming**
 - All signals were padded or trimmed to the same length of 1s
- **Random noise [Sainath-2015]**
 - Added to all signals so that the resulting SNR is in $[-5, 10]$ dB
- **Data augmentation [Park-2019]**
 - SpecAugment: masking random regions of the spectrogram along the temporal and frequency dimensions

[Sainath-2015] T. N. Sainath and C. Parada, “Convolutional neural networks for small-footprint keyword spotting,” in Interspeech, (Dresden, Germany), Sept. 2015.

[Park-2019] D. S. Park et al., “SpecAugment: A simple data augmentation method for automatic speech recognition,” in Interspeech, (Graz, Austria), Sept. 2019.

Preprocessing (2)

- Segmentation

- The audio waveform is split into windows of 25 ms, segmented with a 10 ms time shift (400 audio samples/frame)

- Mel filterbanks

- 26 triangular filters based on the Mel's scale

- MFCC vectors $\mathbf{x}_i$

- Cepstral features extracted
- From 12 (min) to 39 (max) coefficients per audio frame

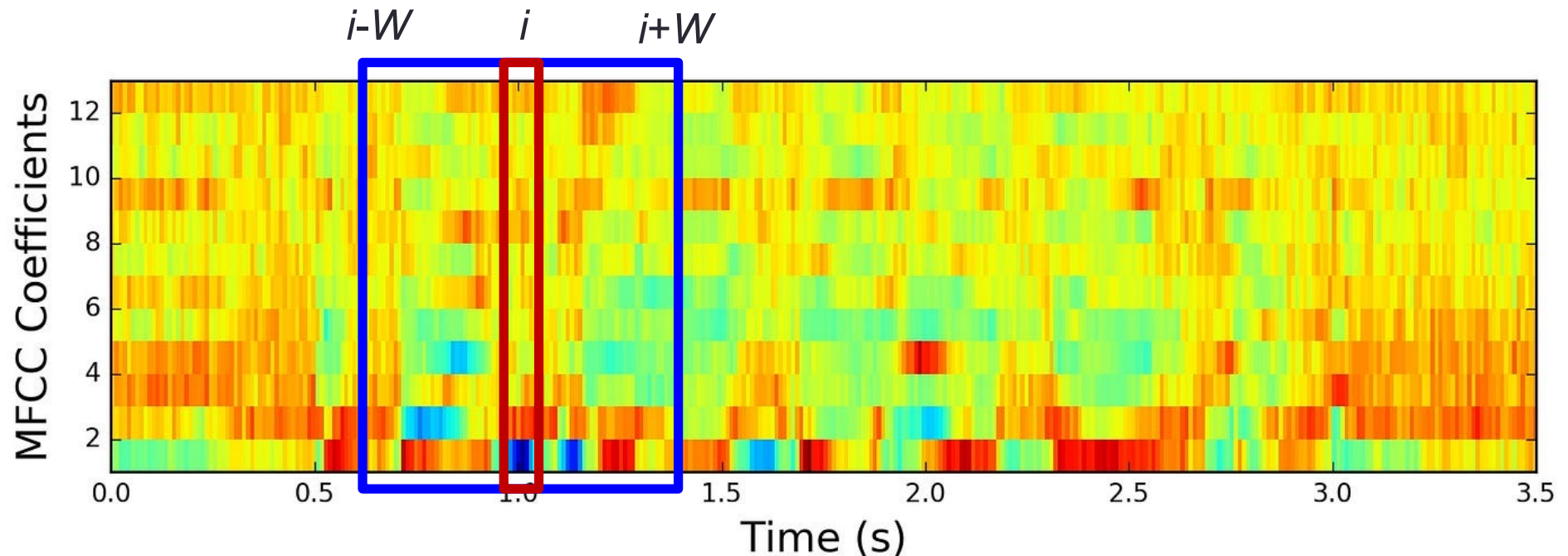
Graph construction (1)

- **Graph nodes**
 - Corresponds to the temporal audio frames (their corresponding MFCC feature vectors), one node for each audio frame
- **Undirected graph (links are bi-directional)**
 - This allows each audio frame to both being influenced and influence its own neighbors, allowing the propagation of information in the graph in both forward and backward time directions
 - Hence, each audio frame **can access contextual information on both past and future audio frames**
- **Graph links**
 - Are unweighted, they are only used to represent **temporal proximity** of audio frames
- **Adjacency matrix A**
 - **Encodes temporal dependencies** using a **window approach**
 - Window W is a hyperparameter (user selected)

Graph construction (2)

Algorithm

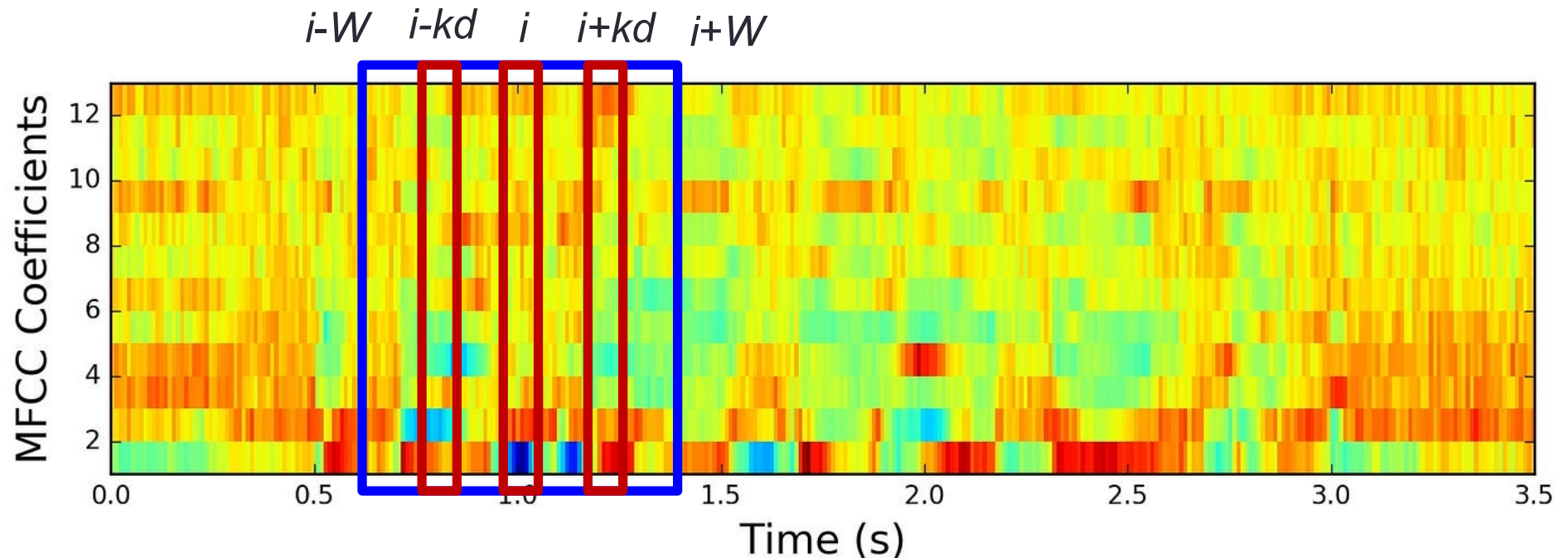
- Create **one graph node for each audio frame**
- **For node $i = 1$ to n**
 - Current audio frame i
 - Connect i with all frames j in $[i-W, i+W]$ (i.e., set $A_{ij}=1$)
- **End for**



Graph construction - dilation d (3)

- Create new adjacency matrix A^d

$$A_{ij}^d = \begin{cases} w_{ij} & j \in \{[i + d, i + d + W] \cup [i - d, i - d - W] \cup j \in \{1, \dots, n\}\} \\ 0 & \text{elsewhere} \end{cases}$$



Graph construction - dilation d (3)

- Create new adjacency matrix A^d

$$A_{ij}^d = \begin{cases} w_{ij} & j \in \{[i + d, i + d + W] \cup [i - d, i - d - W] \cup j \in \{1, \dots, n\}\} \\ 0 & \text{elsewhere} \end{cases}$$

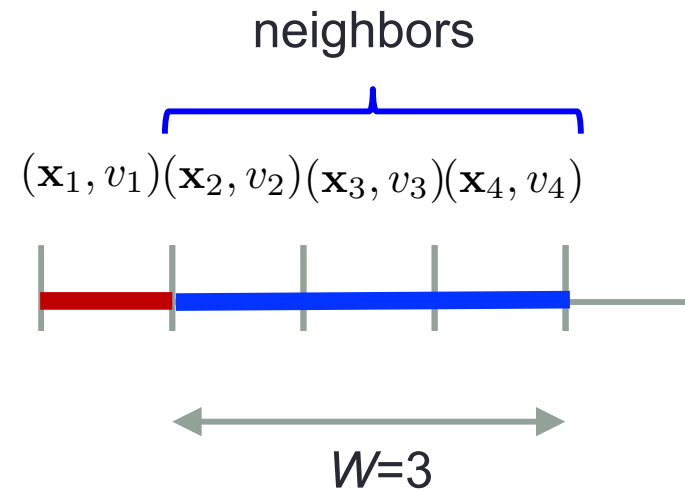
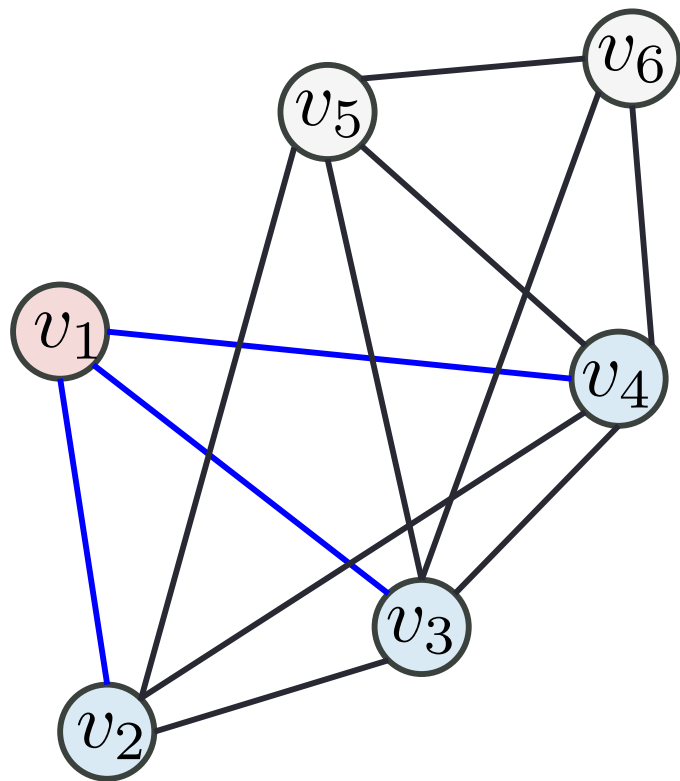
where:

$$w_{ij} = \begin{cases} \frac{\mathbf{x}_i^T \mathbf{x}_j}{\|\mathbf{x}_i\| \|\mathbf{x}_j\|} & \text{cosine window mode} \\ 1 & \text{simple window mode} \end{cases}$$

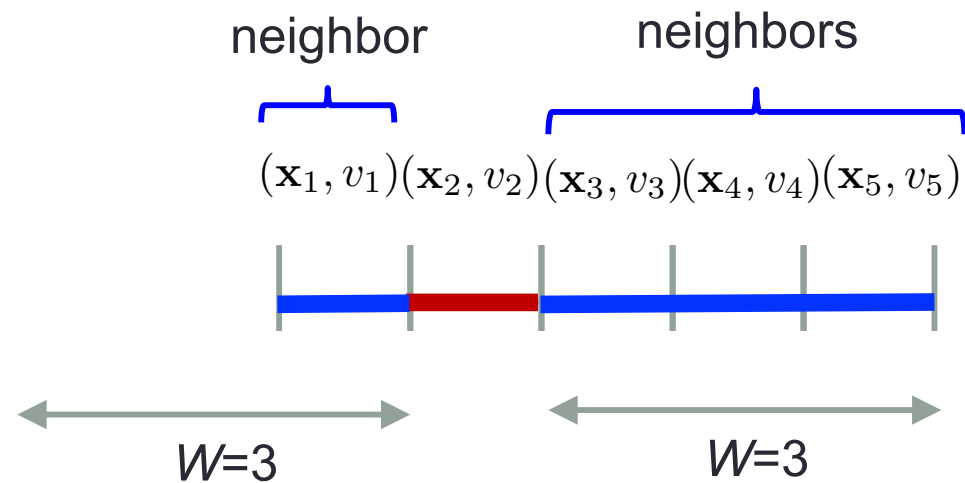
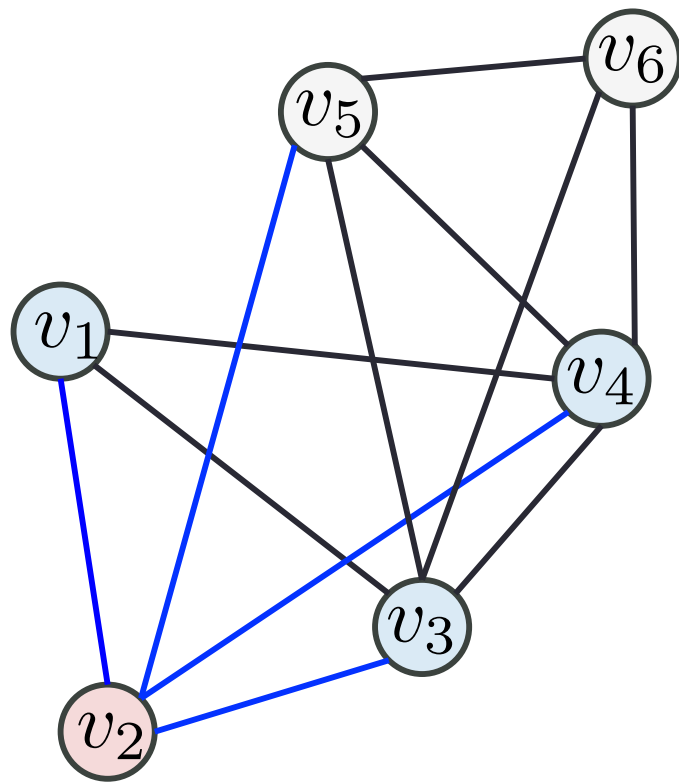
cosine similarity



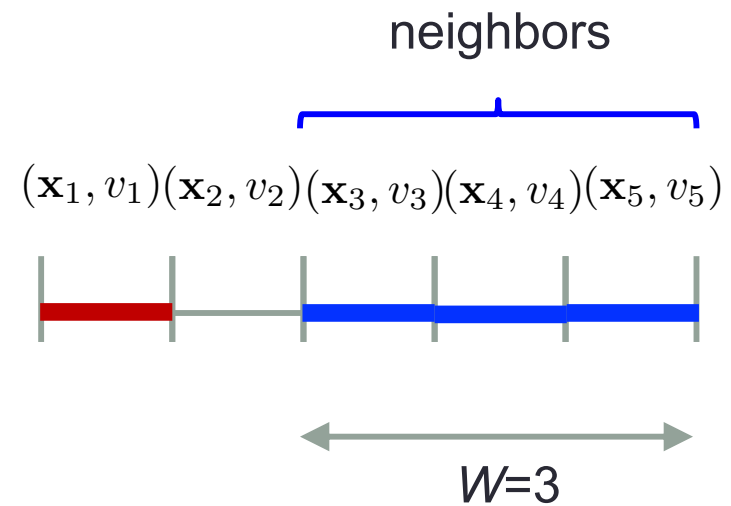
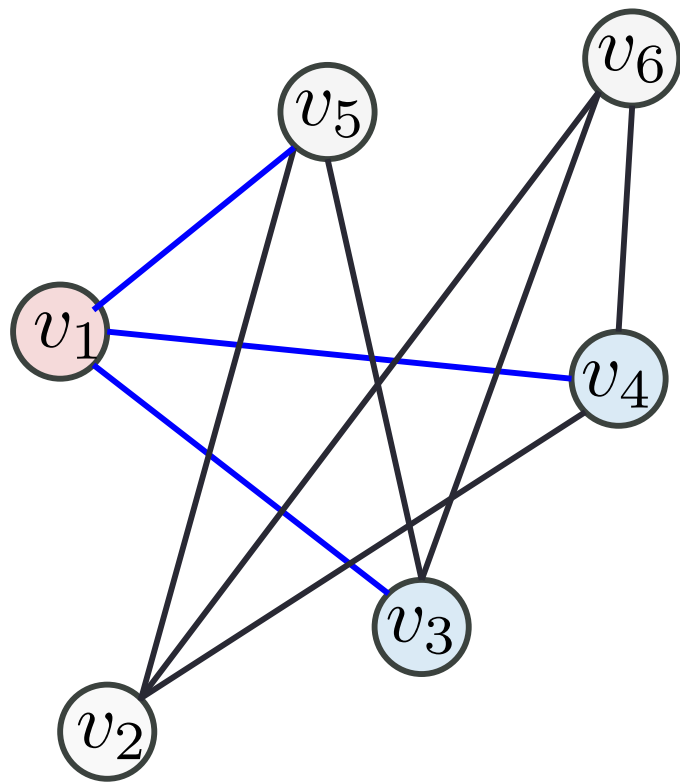
Graph example (1)



Graph example (2)

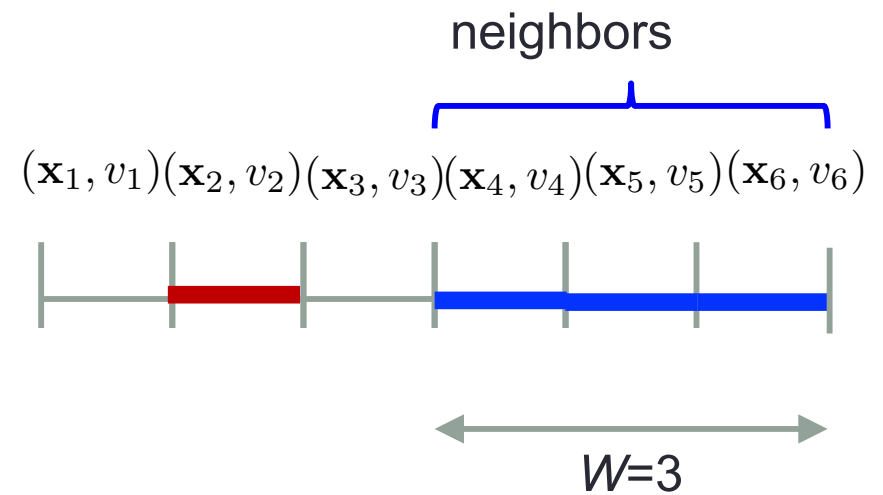
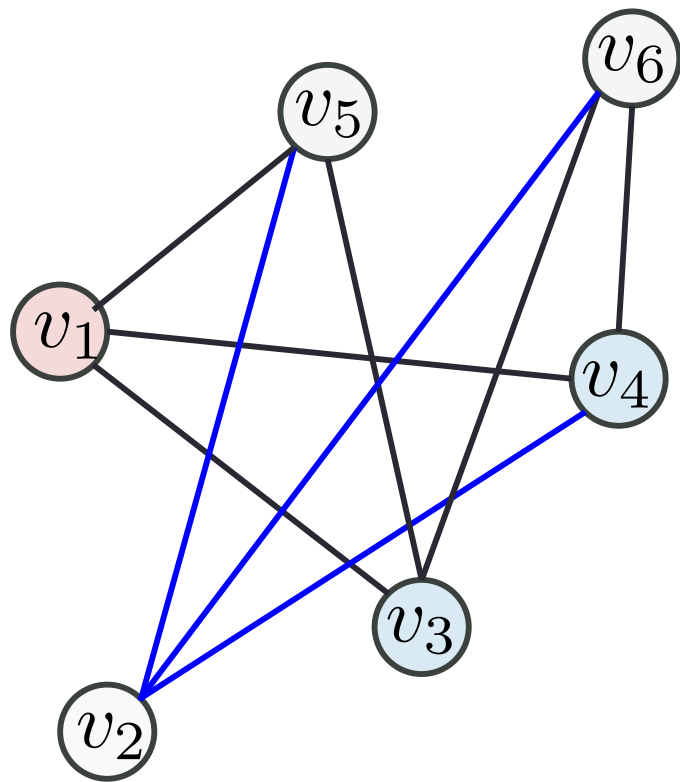


Dilated graph example (1)



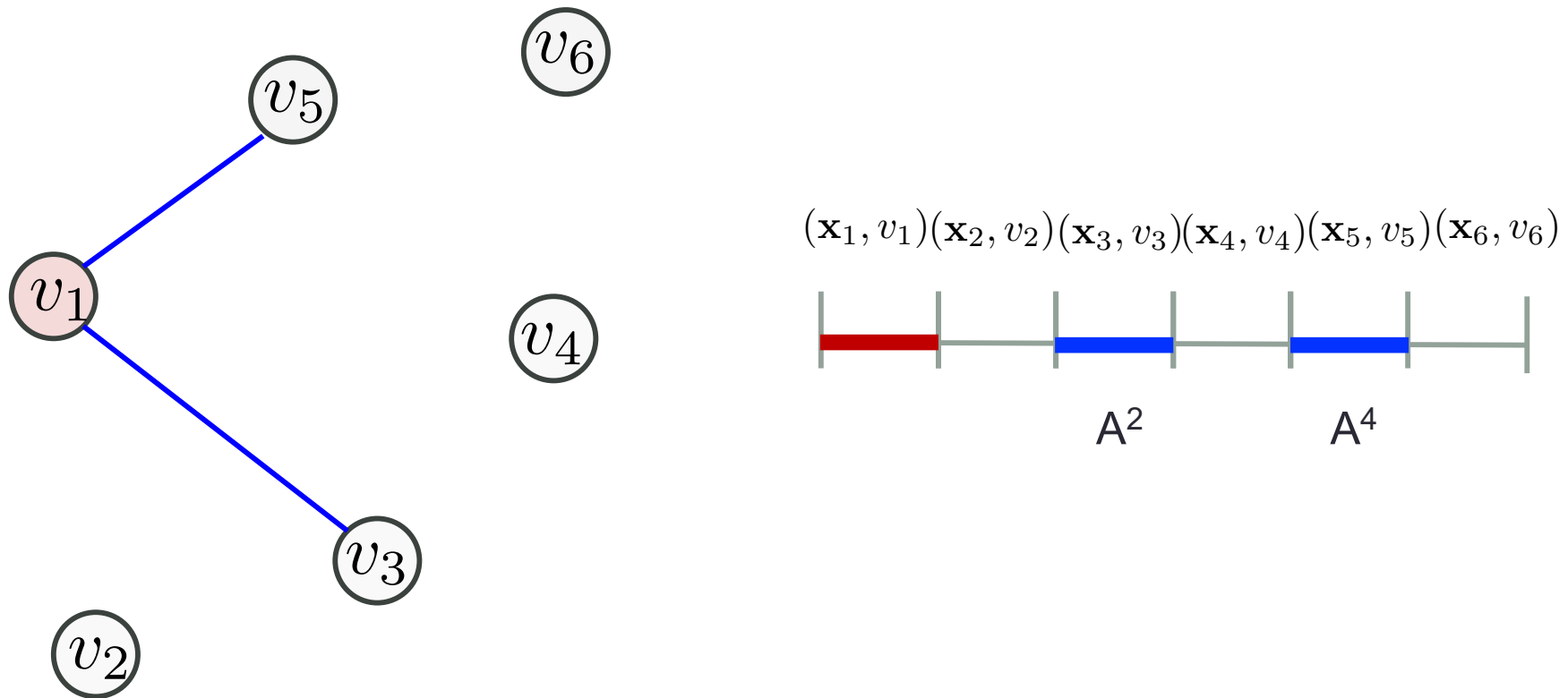
dilation par $d=2$

Dilated graph example (2)



dilation par $d=2$

Multi-resolution via dilation



Example: $W=1$, union of A^2, A^4

GNN

1) Message gathering

Each node collects embedding vectors from its neighbors, according to the graph structure (connected nodes)

2) Message aggregation

The collected embedding vectors are combined through an **aggregation function** and transformed via a dense layer into a fixed-size vector

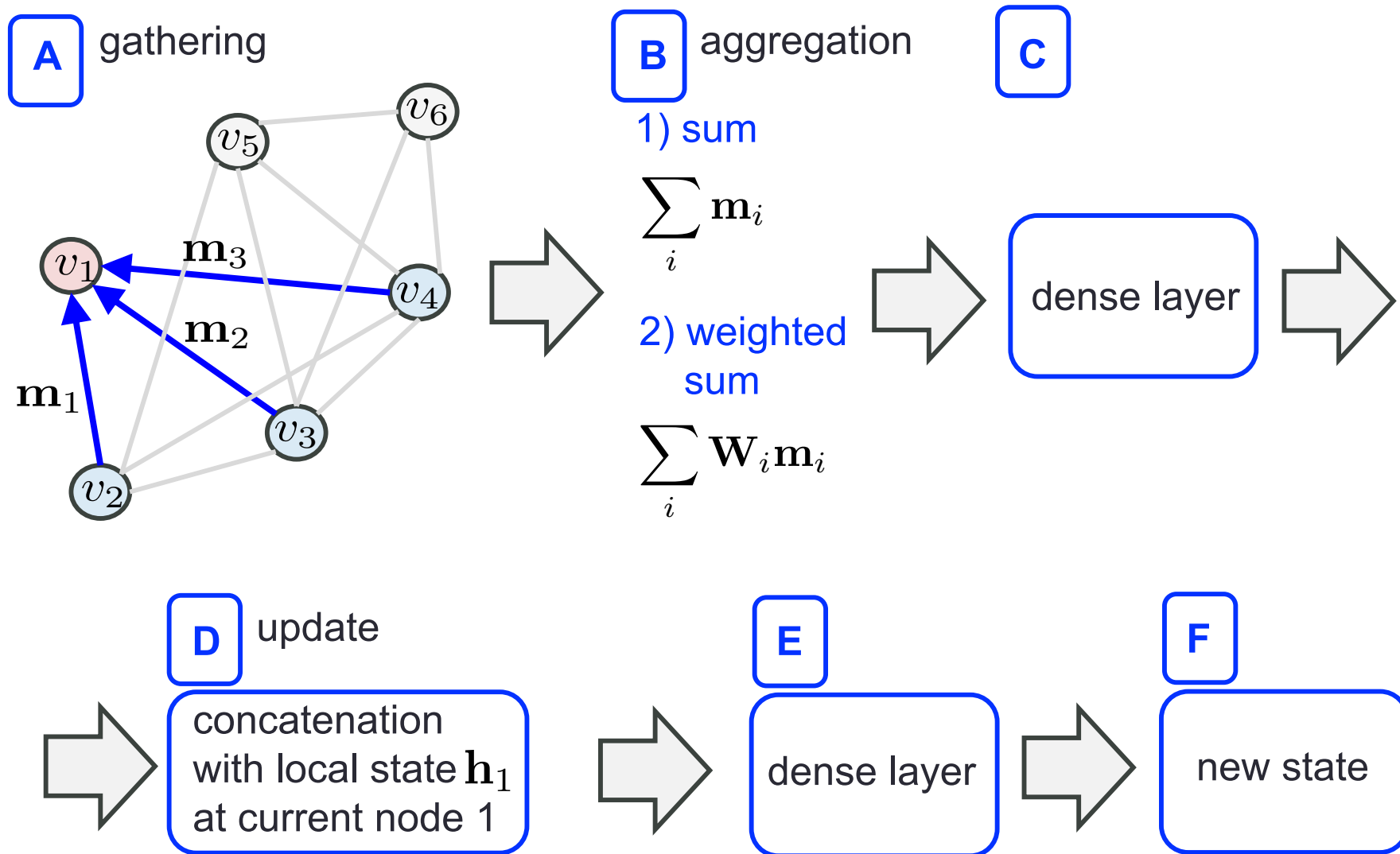
3) Node state update

The node representation vector is updated by **concatenating** its current state with the aggregated information from its neighbor (from step 2). After that, another dense layer is used to obtain the final state vector

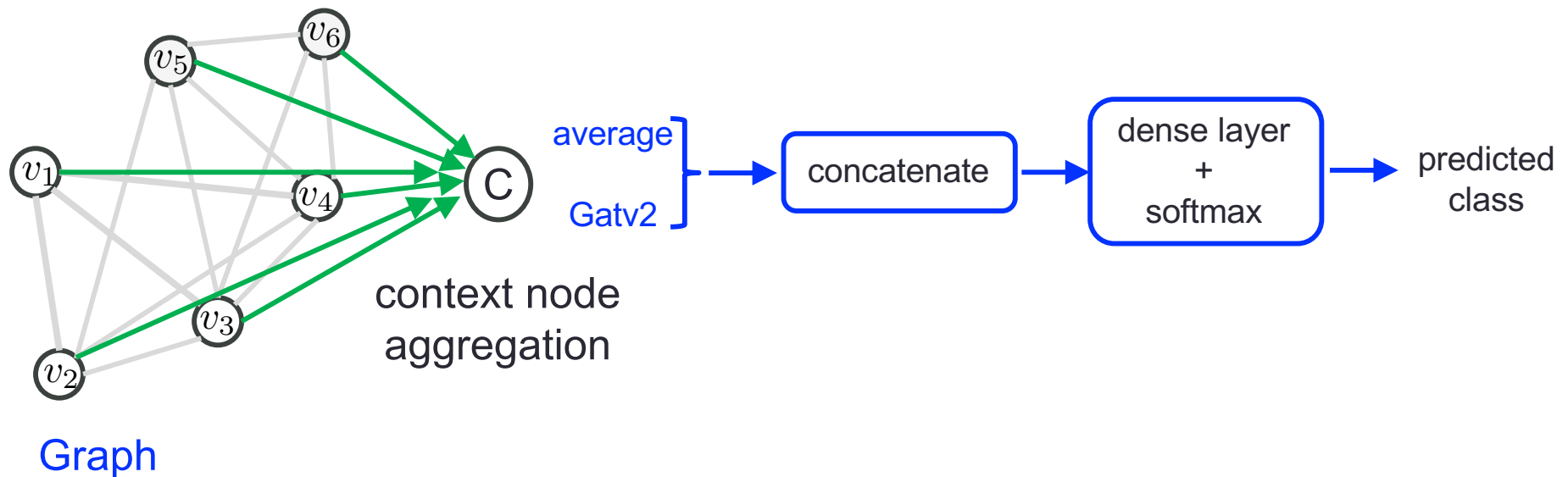
All dense layers include: **20% dropout**, layer **normalization**, **ReLU** activation functions (as the non-linearity)

5 GCN layers were stacked

Message passing pipeline



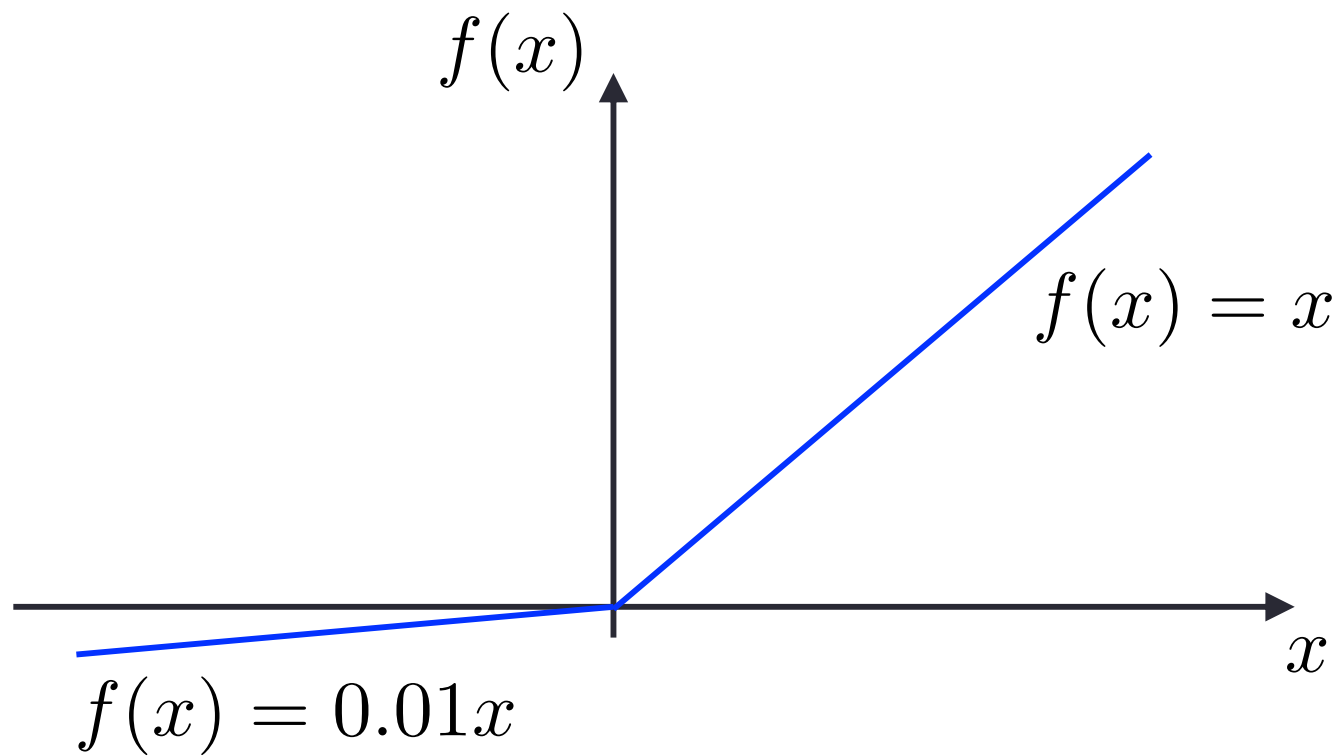
Graph-level task



State vectors (after message passing) **from all nodes** are gathered (pooled) by a context node (C), where they are **1) averaged**, **2) processed by a Gatv2** (attention with dropout) layer

After that, the output of the Gatv2 layer is **concatenated** with the average vector and the concatenated vector goes through a **dense layer** and a final **softmax layer**

Leaky ReLU non-linearity



GATv2 [Brody2022]

$$\mathbf{z}'_u = \sum_{v \in \mathcal{N}(u) \cup \{u\}} \alpha_{uv} \mathbf{W} \mathbf{h}_v$$

$$\alpha_{uv} = \frac{\exp(\mathbf{a}^T \text{LeakyReLU}(\mathbf{W}(\mathbf{h}_u \parallel \mathbf{h}_k)))}{\sum_{k \in \mathcal{N}(u) \cup \{u\}} \exp(\mathbf{a}^T \text{LeakyReLU}(\mathbf{W}(\mathbf{h}_u \parallel \mathbf{h}_k)))}$$

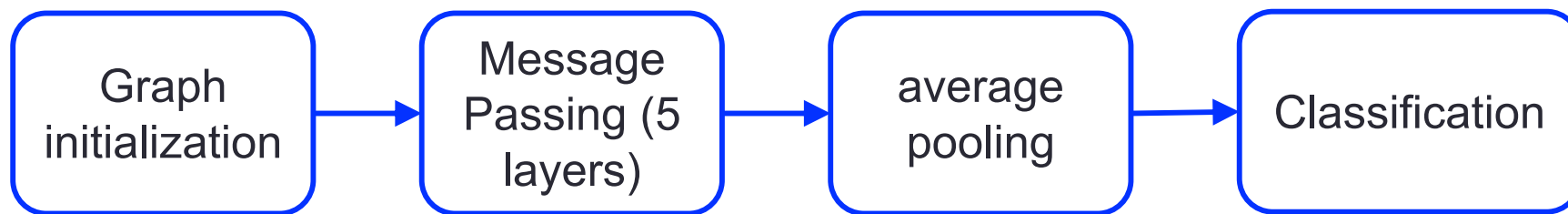
source target

Dropout probability is accounted for the normalized attention coefficients $\rightarrow$ to expose each node u to a **stochastically sampled neighborhood** during training

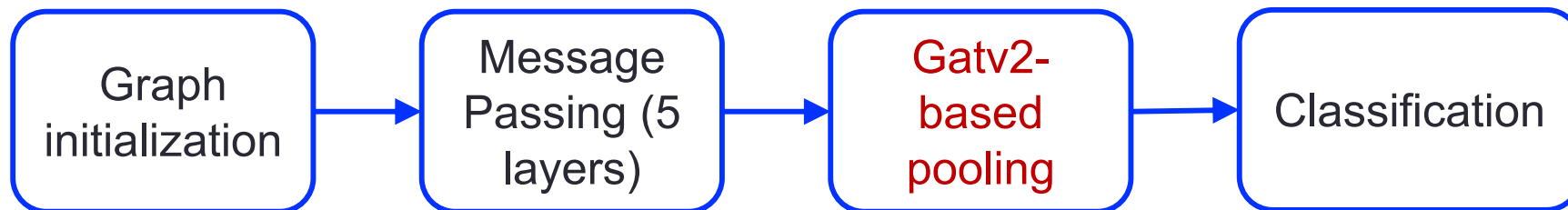
[Brody2022] S. Brody et al., “How attentive are graph attention networks?,” International Conference on Learning Representations (ICLR), 2022.

Alternative pipelines

Simple GNN



GNN+attention



Example results (35 sound classes)

Model	Accuray	Pars	Multiplies	Time [ms]	Memory
Simple GNN W=25, A 0/1 edges	85.57	68k	6.474M	1.41	264kB
Simple GNN W=25 $A^2+A^4+A^8$ 0/1 edges	90.17	68k	6.474M	1.89	264kB
Simple GNN W=25, cosine sim edges	90.21	68k	6.660M	2.12	264kB
GNN+attention W=5 $A^2+A^4+A^6+A^8$ 0/1 edges 5 attention heads	94.58	155k	11.102M	2.30	607kB

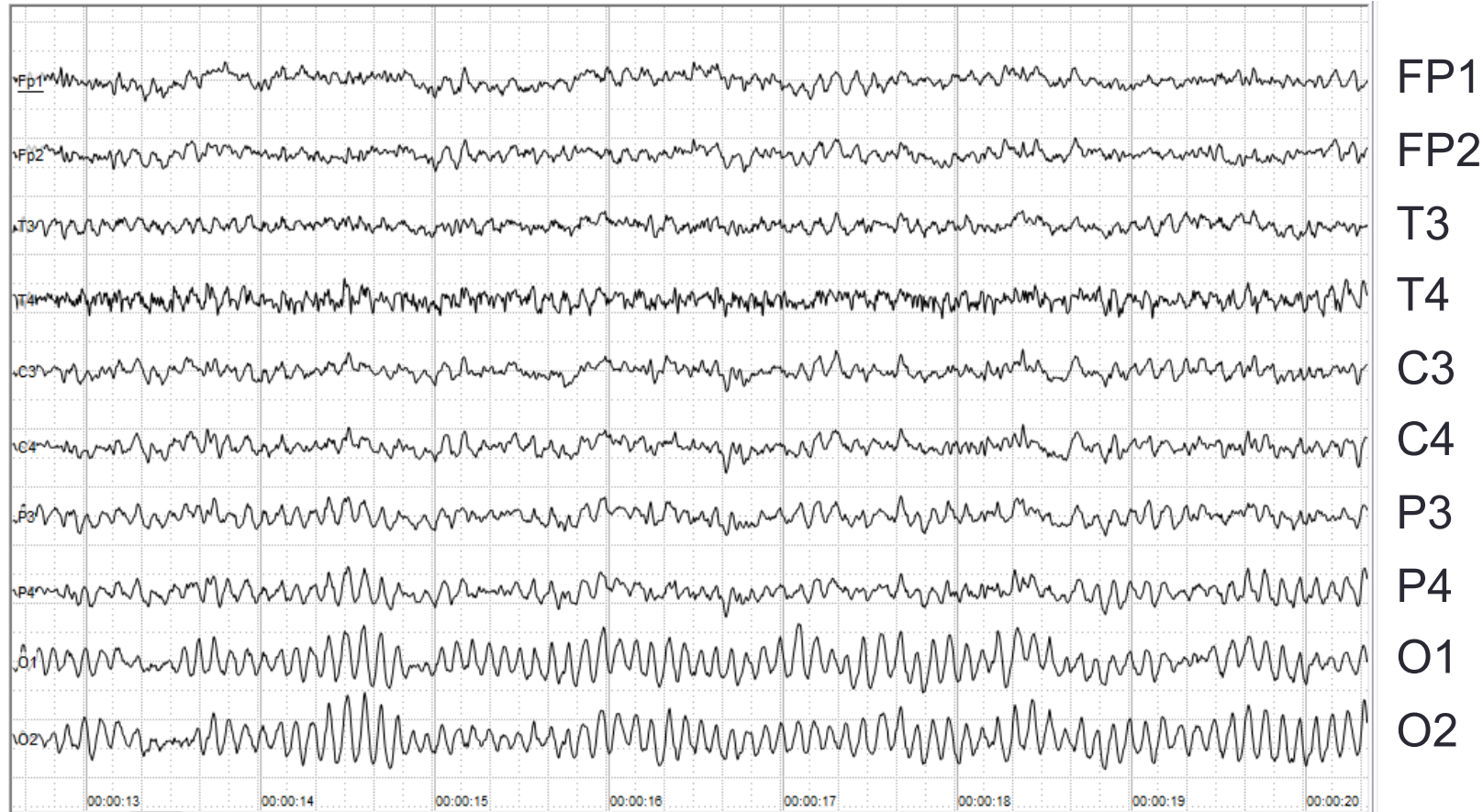
Example application - EEG

Electroencephalogram (EEG) signals are recordings of the brain's electrical activity, captured by electrodes on the scalp that detect the synchronized firing of millions of cortical neurons

These signals appear as wavy lines that are analysed by doctors to help diagnose brain conditions like epilepsy, and they are also used in research for brain-computer interfaces and emotion detection. The signals are characterized by different frequency bands which correspond to different states of brain activity

Multidimensional, spatio-temporal signal

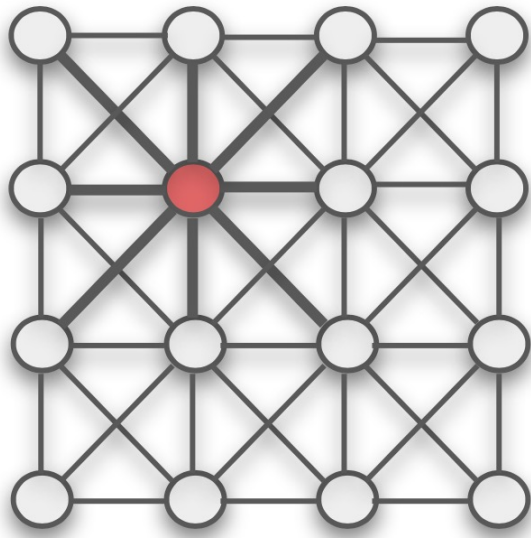
EEG signal



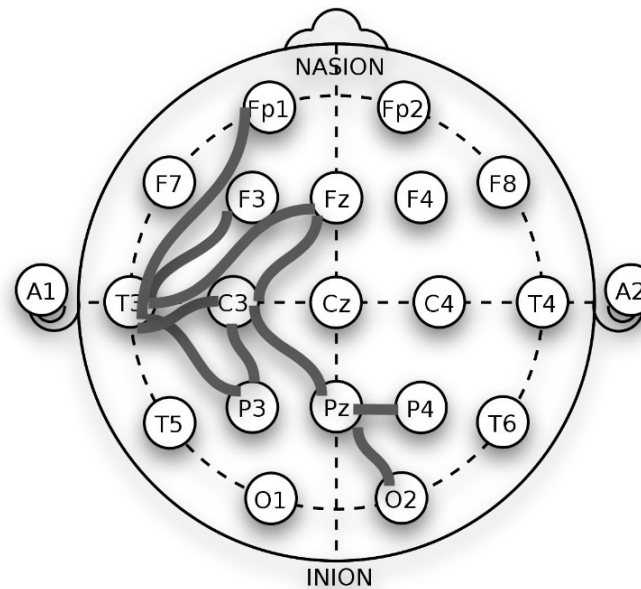
[Demir-2021] Andac Demir et al., “EEG-GNN: Graph Neural Networks for Classification of Electroencephalogram (EEG) Signals,” *International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC)*, 2021.

Sensors mapping vs processing

- Physical location of sensors and representation via CNNs versus GNNs



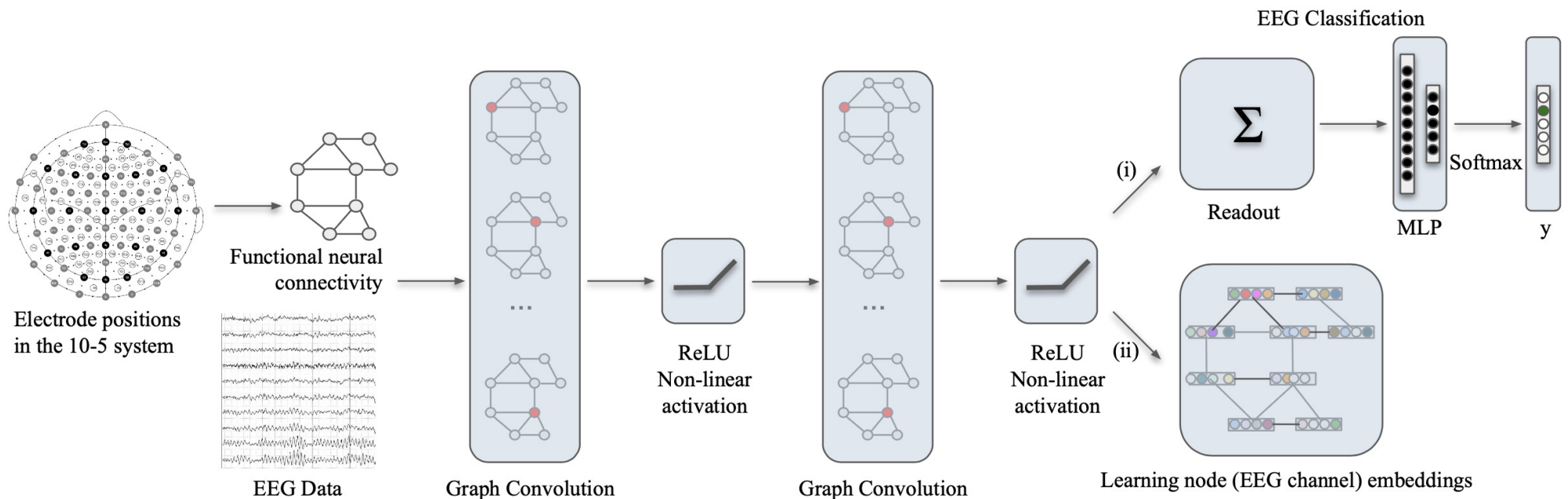
Standard CNN-based representation



A1, A2: "ground" electrodes placed on ears, provide a reference point for measuring electrical activity from the scalp electrodes

Sensor placement and GNN-based representation

Framework



Graph edges can be weighted

- Better representation of “proximity” of sensing nodes

Several message passing steps

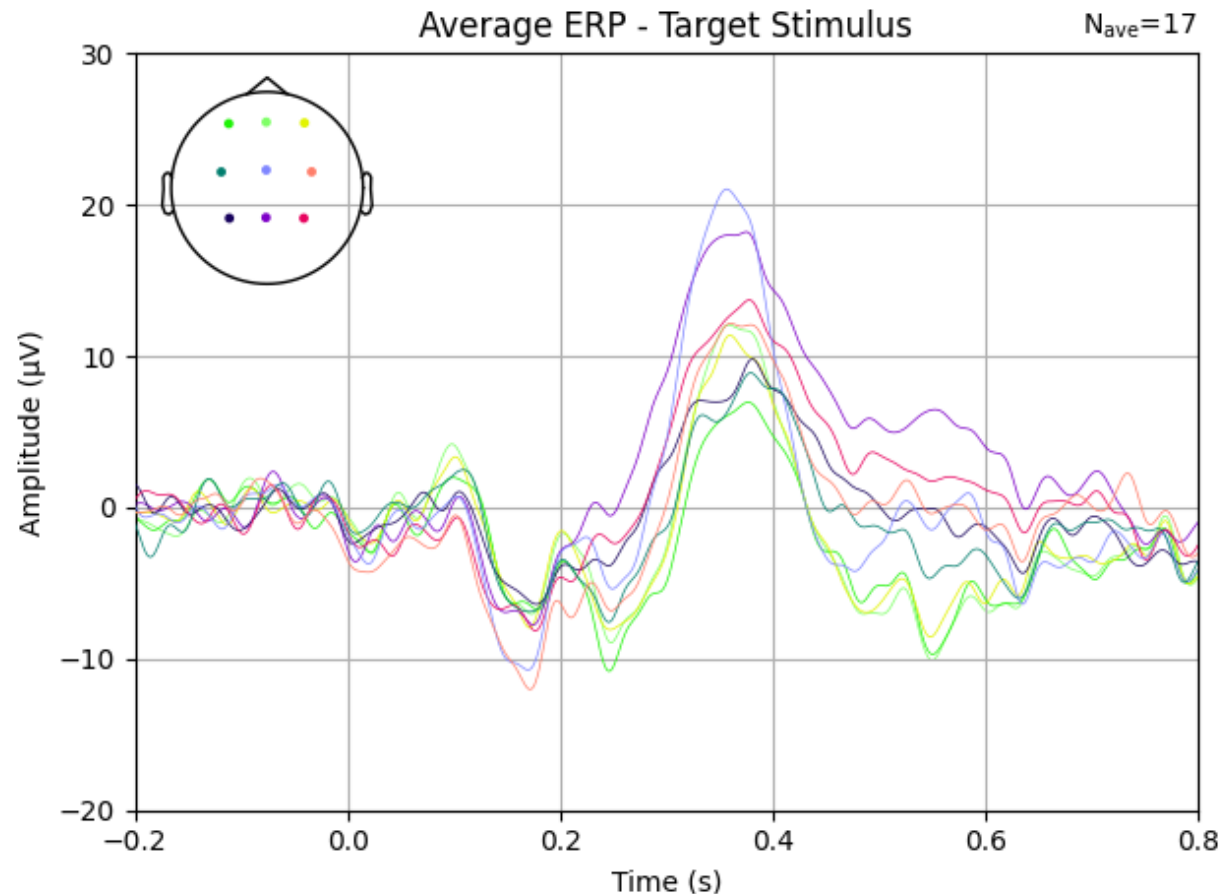
Final graph pooling of global features → classification

P300 response

- Event related potentials (ERP)
 - Characteristic waves that appear in the EEG signal as a response to a specific stimulus or event
- P300 ERP
 - Is a specific ERP that emerges approximately 300 ms after the presentation of a stimulus. It is linked to neural processes like attention allocation, working memory, and decision-making
 - The P300 is mostly evoked in the parietal, frontal, and central lobes
- P300 signal can be elicited by a specific stimulus
 - For example, an image of a cat (non-target stimulus) is frequently shown to the participant
 - Less frequently, the cat image is interrupted by an image of a tree (target stimulus). When the participant is actively engaged in detecting the less frequent target stimulus (the tree), their brain's response to this target stimulus, compared to the non-target stimulus, has measurable differences, which can be assessed

P300 response

- The P300 response is a surge in the signal amplitude (parietal lobe) appearing approx. 300ms after the stimulus



P300 speller

- A P300 speller is one of the most researched types of brain spellers and makes use of this characteristic P300 ERP component to estimate what letter a subject is thinking about sequentially in order to spell out a word
- First P300 speller: due to Farewell and Donchin in 1988
 - It consisted of a 6 by 6 matrix called the “Matrix Speller” that contained all 26 English letters and the numbers 1–9. The subject in the experiment would be asked to direct their attention to a target letter, and then a row or column would be randomly and sequentially flashed from the matrix that may or may not contain the target letter
 - A strong P300 brain signal was typically associated with each time the target letter flashed. Because the experimenters knew which row or column had been flashed during each of those events, could deduce what the target letter was by combining the rows and columns that elicited a P300 response

Farewell and Donchin P300 Speller

- Initial matrix with flashing (a) rows & columns, which was later modified with (b) random combinations of letters

(a)

CRT		N/A		N/A	
BCI		B			
A	B	C	D	E	F
G	H	I	J	K	L
M	N	O	P	Q	R
S	T	U	V	W	X
Y	Z	1	2	3	4
5	6	7	8	9	-

(b)

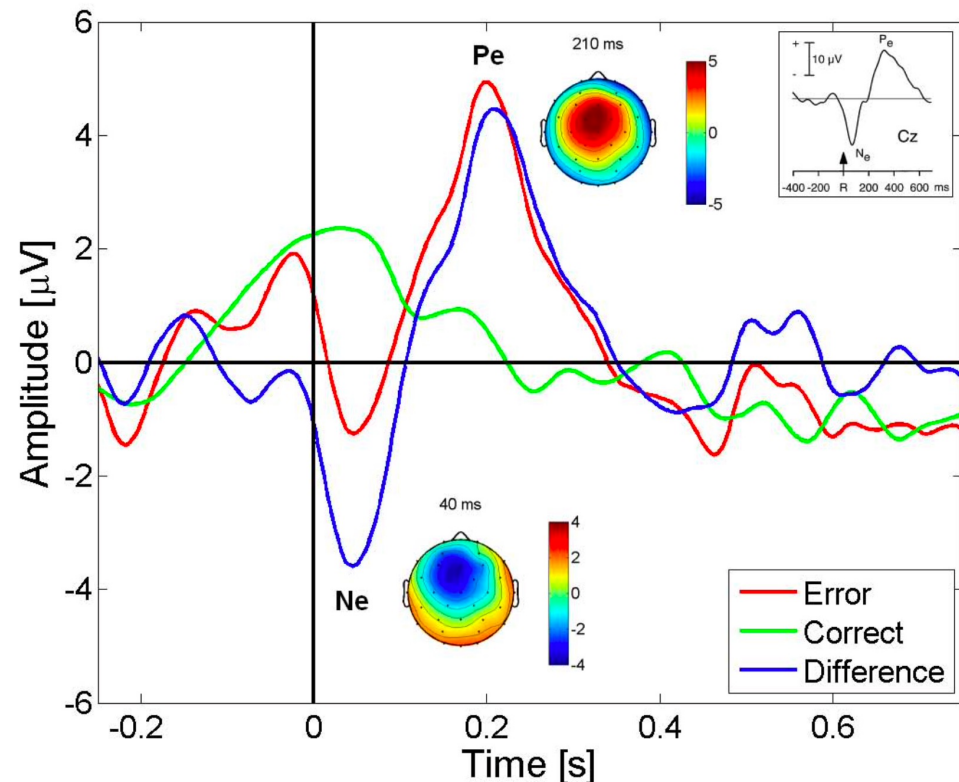
17" LCD		50 Hz		1280x1024 px		55 cm	
BCI		B					
A	B	C	D	E	F		
G	H	I	J	K	L		
M	N	O	P	Q	R		
S	T	U	V	W	X		
Y	Z	1	2	3	4		
5	6	7	8	9	-		

Matrix speller

Error Potentials (ErrP)

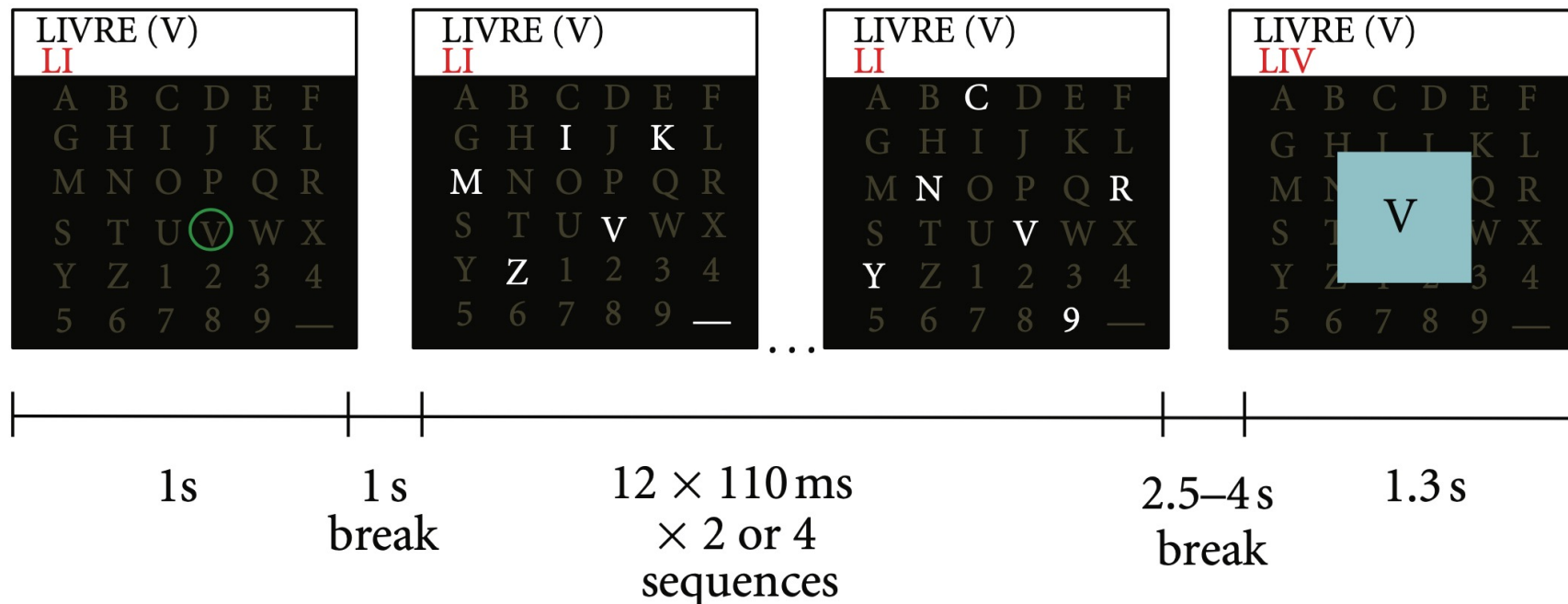
- ErrP
 - Are alterations of the EEG trace following the subject's perception of **erroneous feedback** (i.e., when a person makes an error, or perceives an error is made by another person or an agent)

Example of correct (green) vs error (red) response signals



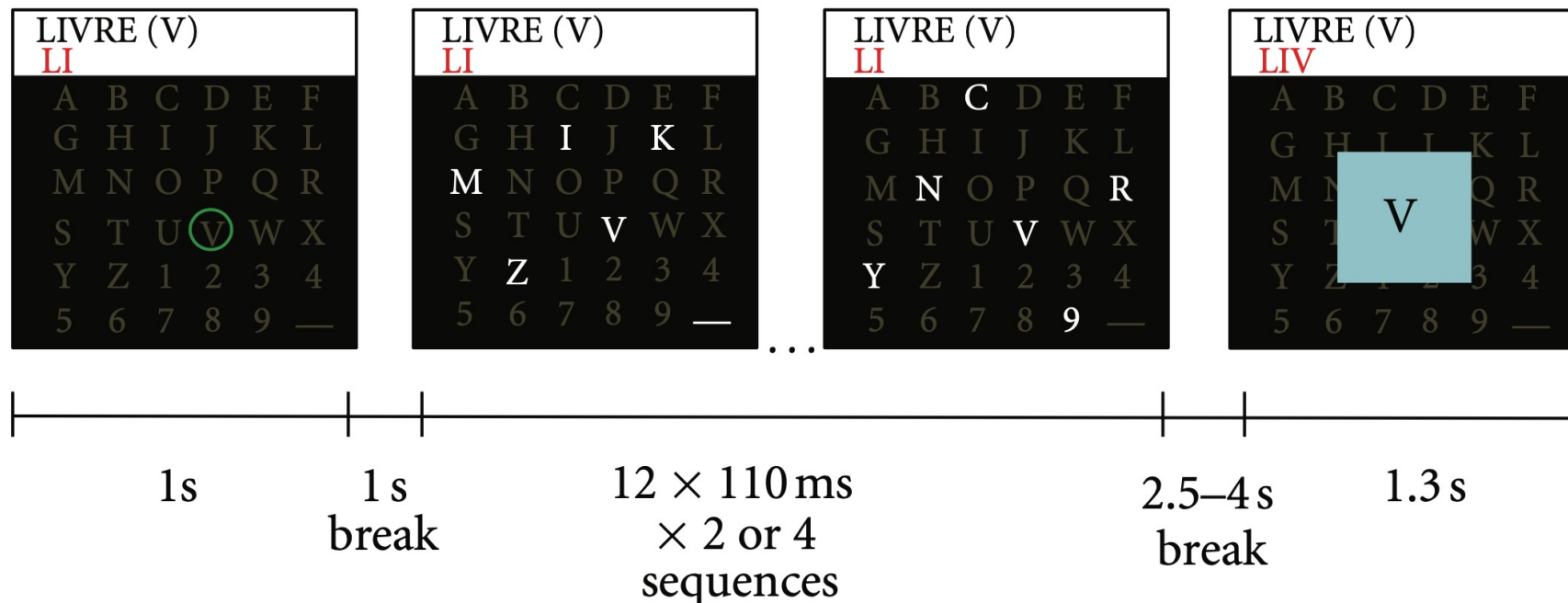
P300 Copy Spelling task

- Instruct the user to focus on a **target letter** (**V** in the below figure)
- The target letter is initially indicated to the user with a circle for 1s
- Group of letters (random matrix) are flashed on screen every 110ms
- The P300 signal is detected to classify the user selection
- The selected letter is visualized in the center of the screen for 1.3s



P300 Copy Spelling task

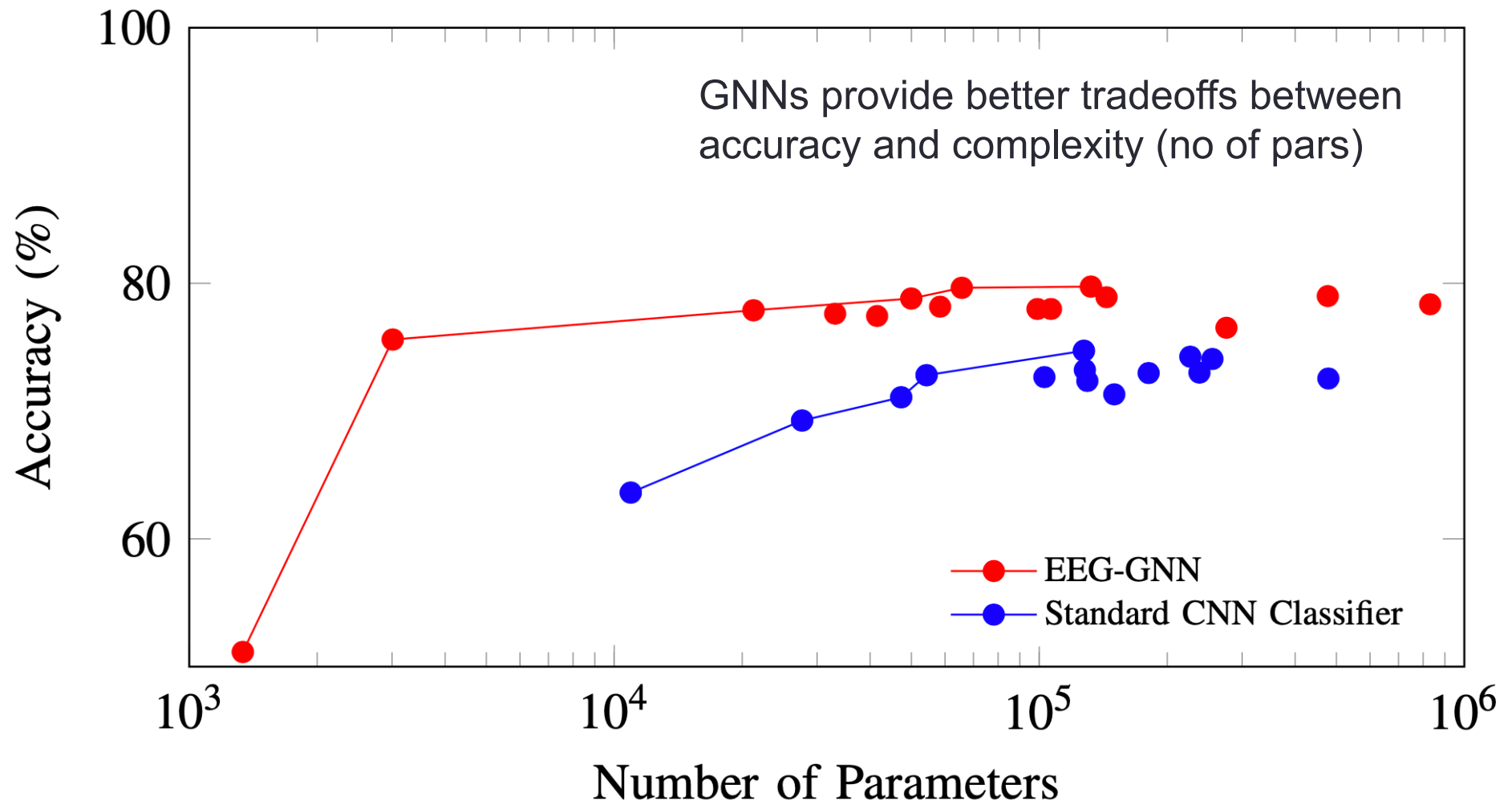
- In case the P300 detector **makes a mistake**
- A letter **different from the initial target** will be visualized at the end
- An **ErrP signal** should appear in the user EEG trace
- The detection of the ErrP allows for **error correction** (e.g., modifying the selection with the second guess from the classifier)



CNN vs GNN

- Assessment in [Demir-2021]
- Detection of ErrP potentials to improve the accuracy of a P300 spelling task
- EEG data recorded at downsampled rate of 200Hz from 56 channels
- 16 healthy subjects involved in the experiments
- Each subject performed 340 trials
- If there is an inconsistency between the subject intention (target letter) and the outcome of the BCI system → an elicited ErrP potential should be detected
- Binary class labels: erroneous or correct feedback

CNNs vs GNNs (Pareto frontier)



EEG references

[Ferrez-2007] P.W. Ferrez “Error-related EEG potentials in brain computer interfaces,” *PhD thesis*, EPFL, 2007.

[Margaux-2012] P. Margaux et al., “Objective and Subjective Evaluation of Online Error Correction during P300-Based Spelling,” *Advances in Human-Computer Interaction*, 2012.

[Demir-2021] A. Demir et al., “EEG-GNN: Graph Neural Networks for Classification of Electroencephalogram (EEG) Signals,” *EMBC*, 2021.

GRAPH NEURAL NETWORKS

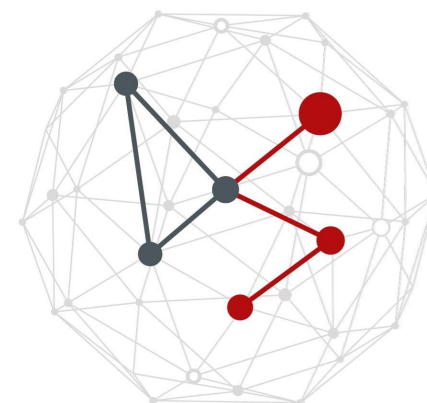
Michele Rossi

michele.rossi@unipd.it

Dept. of Information Engineering

Dept. of Mathematics

University of Padova, IT



APPENDIX A

Chebyshev polynomials

- Proof of the following equality

$$\sum_{i=0}^d w_i \mathbf{U} T_i(\mathbf{\Lambda}) \mathbf{U}^T = \sum_{i=0}^d w_i T_i(\mathbf{L})$$

- We prove this **by induction**,
 - by first assessing that it holds for $i=0$ and $i=1$
 - and subsequently verifying that it holds for $i \geq 1$

Chebyshev polynomials

- For $i=0$, we have (by construction)

$$T_0(\mathbf{\Lambda}) = \mathbf{I}$$

- and it also holds

$$\mathbf{U}T_0(\mathbf{\Lambda})\mathbf{U}^T = \mathbf{U}\mathbf{U}^T = \mathbf{I}$$

- and, by construction

$$T_0(\mathbf{L}) = \mathbf{I}$$

- Hence, it holds that

$$\mathbf{U}T_0(\mathbf{\Lambda})\mathbf{U}^T = T_0(\mathbf{L})$$

Chebyshev polynomials

- For $i=1$, we have (by construction)

$$T_1(\mathbf{\Lambda}) = \mathbf{\Lambda}$$

- from which, it descends that

$$\mathbf{U}T_1(\mathbf{\Lambda})\mathbf{U}^T = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$$

- and, by construction

$$T_1(\mathbf{L}) = \mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$$

- Hence, it holds that

$$\mathbf{U}T_1(\mathbf{\Lambda})\mathbf{U}^T = T_1(\mathbf{L})$$

Chebyshev polynomials

- By induction, we assume that (inductive hypothesis)

$$\mathbf{U}T_i(\mathbf{\Lambda})\mathbf{U}^T = T_i(\mathbf{L}), \text{ for } i \text{ and } i-1, i \geq 1$$

- we need to prove that

$$\mathbf{U}T_{i+1}(\mathbf{\Lambda})\mathbf{U}^T = T_{i+1}(\mathbf{L})$$

- Since

$$T_{i+1}(\mathbf{\Lambda}) = 2\mathbf{\Lambda}T_i(\mathbf{\Lambda}) - T_{i-1}(\mathbf{\Lambda})$$

- It also holds

$$\mathbf{U}T_{i+1}(\mathbf{\Lambda})\mathbf{U}^T = 2\mathbf{U}\mathbf{\Lambda}T_i(\mathbf{\Lambda})\mathbf{U}^T - \mathbf{U}T_{i-1}(\mathbf{\Lambda})\mathbf{U}^T$$

Chebyshev polynomials

Starting from this result

$$\mathbf{U}T_{i+1}(\mathbf{\Lambda})\mathbf{U}^T = 2\mathbf{U}\mathbf{\Lambda}T_i(\mathbf{\Lambda})\mathbf{U}^T - \mathbf{U}T_{i-1}(\mathbf{\Lambda})\mathbf{U}^T$$

Since,

$$\mathbf{U}^T\mathbf{U} = \mathbf{I}$$

We can rewrite it as (due to the inductive hypothesis)

$$\begin{aligned}\mathbf{U}T_{i+1}(\mathbf{\Lambda})\mathbf{U}^T &= 2\mathbf{U}\mathbf{\Lambda}\mathbf{U}^T\mathbf{U}T_i(\mathbf{\Lambda})\mathbf{U}^T - \mathbf{U}T_{i-1}(\mathbf{\Lambda})\mathbf{U}^T \\ &= 2\mathbf{L}T_i(\mathbf{L}) - T_{i-1}(\mathbf{L}) = T_{i+1}(\mathbf{L})\end{aligned}$$

QED